

Mobil Geliştiriciler İçin GraphQL

Ömer Faruk Yavuz

Aralık 2025

GraphQL Nedir?

GraphQL, API'ler için kullanılan bir **sorgu dili** ve bu sorguları çalıştıran **sunucu tarafı bir çalışma zamanı (runtime)**dır. Temel yaklaşım şu şekilde özetlenebilir:

- İstemci (mobil uygulama) **tam olarak hangi alanlara ihtiyaç duyduğunu** bildirir.
- Sunucu, ızca talep edilen alanları geri döndürür.

GraphQL'in başlıca özellikleri şu şekildedir:

- Belirli bir veritabanına veya depolama motoruna bağımlı değildir; mevcut arka uç (backend) kodunun ve verinin üzerine konumlandırılır.
- 2015 yılında açık kaynak olarak yayımlanan bir spesifikasyona dayanır.
- Pek çok programlama dili ve ekosistem için hem sunucu (*server*) hem de istemci (*client*) kütüphaneleriyle desteklenmektedir.

Klasik REST yaklaşımında “şu endpoint’e git, ne verirse al” anlayışı ağırlıktayken, GraphQL’de istemci “*yalnızca şu alanları gönder*” diyerek çok daha kontrollü veri çekimi sağlayabilir.

REST ve GraphQL Arasındaki Temel Farklar

REST Yaklaşımı

REST tabanlı API'lerde çoğunlukla her kaynak türü için ayrı endpoint'ler tanımlanır:

- /users
- /users/5
- /users/5/posts

Bu modelin mobil tarafta sık karşılaşılan sorunları aşağıdaki gibidir:

Over-fetching İlgili ekran için gerekli olandan **çok daha fazla** alanın istemciye gönderilmesi-dir.

Under-fetching Tek bir istekle ihtiyaç duyulan tüm verinin elde edilememesi ve **ek endpoint çağrılarına** ihtiyaç duyulmasıdır.

Çoklu istek gecikmesi Her REST çağrısı:

- Ayrı bir HTTP isteği,
- Ayrı bir ağ (network) gidiş-dönüş süresi (round-trip),
- Ayrı bir JSON çözümleme (parse) maliyeti anlamına gelmektedir.

Özellikle hücresel ağların kullanıldığı mobil ortamlarda bu ek istekler, belirgin düzeyde **gecikmeye (latency)** ve **pil tüketiminin artmasına** neden olur.

GraphQL Yaklaşımı

GraphQL servisleri genellikle tek bir endpoint üzerinden hizmet verir:

/graphql

İstemci, bu endpoint'e **GraphQL sorgusunu (query)** HTTP isteğinin gövdesinde iletir. Örneğin:

POST /graphql

```
{
  "query": "query { users { id name email } }"
}
```

Sunucu, öncelikle sorguyu tanımlı şemaya göre **doğrular**, ardından arka planda çalışan resolver fonksiyonlarını çağırarak sonucu üretir ve yanıtı JSON formatında döndürür.

Bu model ile:

- İhtiyaç duyulan **alanlar tek tek seçilebilir**.
- Birden fazla REST isteği ile elde edilebilecek ilişkili veriler **tek bir GraphQL sorgusunda** bir araya getirilebilir.
- Gönderilen ve paketlenen veri miktarı (örneğin 1 MB yerine 100 KB) **çok daha sıkı biçimde kontrol edilebilir**.

Tip Sistemi ve Şema (Schema)

GraphQL, API'nin **tipler** ve **alanlar** üzerinden açık biçimde tanımlanmasını gerektirir. Bu tanımlamaya *schema* adı verilir.

Örneğin, oturum açmış kullanıcının adını döndüren basit bir şema şu şekilde olabilir:

```
type Query {  
  me: User  
}  
  
type User {  
  name: String  
}
```

Bu tiplerin arkasında, her alan için veriyi sağlayan **resolver** fonksiyonları bulunmaktadır.

Sadece İhtiyaç Duyulan Verinin İstenmesi

Sunucu başlatıldıktan sonra istemciler GraphQL sorguları gönderebilir. Örneğin:

```
{  
  me {  
    name  
  }  
}
```

Bu sorgunun olası JSON yanıtı aşağıdaki gibi olacaktır:

```
{  
  "data": {  
    "me": {  
      "name": "Luke Skywalker"  
    }  
  }  
}
```

Burada dikkat edilmesi gereken hususlar şunlardır:

- Sorgunun yapısı ile dönen verinin yapısı **bire bir örtüşmektedir**.
- İstemci yalnızca name alanını talep ettiği için, yanıtta başka herhangi bir alan yer almamaktadır.

Mobil bakış açısıyla bu durum şu sonuçları doğurur:

- İlgili ekran için gereksiz olan alanlar (address, settings, logs vb.) sunucu tarafından hiç gönderilmez.
- Toplam veri boyutunu, örneğin 1 MB düzeyinden 100–200 KB seviyesine indirmek mümkün hale gelir.

Mobil Perspektiften Performans: 1 MB Yerine 100 KB

REST Senaryosu (Örnek)

Bir mobil uygulamanın ana ekranı (*Home*) göz önünde bulundurulsun. Bu ekranda:

- Kullanıcının temel bilgileri,
- Öne çıkan birkaç içerik (örneğin romantik mekânlar),
- Küçük bir promosyon alanı

gösterildiği varsayalım.

REST ile tipik istekler şu şekilde olabilir:

- `/me` → kullanıcı bilgileri (~ 150 KB)
- `/spots` → tüm spot'ların tam içerikleri (~ 800 KB)
- `/promotions` → promosyon bilgileri (~ 100 KB)

Bu durumda toplamda:

- Yaklaşık ~ 1 MB veri,
- 3 ayrı HTTP isteği,
- 3 ayrı ağ gecikmesi ve JSON çözümleme maliyeti söz konusu olur.

Mobil ağ kullanıldığında bu, daha uzun bekleme süreleri, daha yüksek veri tüketimi ve artan pil kullanımı anlamına gelmektedir.

Aynı Senaryo GraphQL ile

Aynı ekran için gerçekte yalnızca aşağıdaki bilgilere ihtiyaç duyulduğu varsayalım:

- Kullanıcı için: `id`, `gender`, `language`
- Spotlar için: `id`, `title`, `thumbnailUrl`, birkaç `vibe` etiketi
- Promosyonlar için: `id`, `title`, `ctaText`

Bu durumda kullanılabilecek örnek bir GraphQL sorgusu şu şekildedir:

```
query HomeScreen {  
  me {  
    id  
    gender  
    language  
  }  
}
```

```
featuredSpots(limit: 5) {  
  id  
  title  
  thumbnailUrl  
  vibes  
}  
  
promotions(limit: 2) {  
  id  
  title  
  ctaText  
}  
}
```

Bu yaklaşımın sonucunda:

- Yalnızca tek bir HTTP isteği gönderilir.
- Yalnızca gerekli alanları içeren görece küçük bir JSON yanıt elde edilir.
- GZip/Brotli gibi sıkıştırma yöntemleriyle birlikte, 1 MB yerine **100–200 KB** aralığında bir payload boyutuna ulaşmak çoğu senaryoda mümkündür.

Önemli Not: GraphQL Sihirli Bir Çözüm Değildir

GraphQL, tek başına tüm veri hacmini otomatik olarak küçültmez; asıl kontrol şema ve sorgu tasarımındadır:

- Sorgu içerisine çok fazla ilişki, çok sayıda alan ve binlerce öge dâhil edilirse, yanıt boyutu yine önemli ölçüde artabilir.
- Performans açısından:
 - Her ekran için mantıklı ve sınırlı kapsamlı sorgular tasarlamak,
 - **Sayfalama** (`limit`, `offset`, `cursor`) kullanmak,
 - Liste ekranlarında önce *hafif* alanları, detay ekranlarında ise daha ağır alanları çağırmak

önerilen yaklaşımlar arasındadır.

API'yi Versiyonlamadan Evrimleştirmek

İstemci gereksinimleri zaman içerisinde değişmektedir. GraphQL'in en önemli avantajlarından biri, API'nin **yeni sürümler tanımlamaya gerek kalmaksızın** evrimleşmesine olanak tanımasıdır.

Örneğin başlangıçta yalnızca name alanına sahip bir User tipi tanımlanmış olsun:

```
type User {  
  name: String  
}
```

Daha sonra, istemci tarafında ortaya çıkan yeni bir gereksinim kapsamında daha ayrıntılı ad alanları tanımlanmak istenebilir:

```
type User {  
  fullName: String  
  nickname: String  
  name: String @deprecated(reason: "Use `fullName`.")  
}
```

Bu durumda:

- name alanı @deprecated ile işaretlenmiştir; böylece yeni istemcilerin fullName alanını kullanması teşvik edilir.
- İstemci tarafında kullanılan araçlar (örneğin IDE eklentileri) geliştiricilere uyarı gösterir.
- Eski alanın artık hiçbir istemci tarafından kullanılmadığından emin olduğunda, bu alan şemadan tamamen kaldırılabilir.

Bu sayede, REST tabanlı mimarilerde sıkça karşılaşılan /v1, /v2 benzeri ağır versiyonlama süreçleri yerine, şema düzeyinde **kademeli bir evrim** sağlanmış olur.

Mobil Geliştiriciler İçin Önerilen Uygulamalar

Mobil senaryolarda GraphQL kullanılırken dikkat edilmesi önerilen başlıklar aşağıda özetlenmiştir:

- **Ekran bazlı sorgular tasarlamak:** Her ekran için mantıklı ve sınırlı kapsamlı bir GraphQL sorgusu tanımlanmalıdır. Aşırı genel, tüm veriyi döndürmeye çalışan tek bir *super-query* tasarımından kaçınılmalıdır.
- **Over-fetching'i azaltmak:** Yalnızca ilgili ekranda gerçekten kullanılan alanların istenmesi, gereksiz alanların sorgulardan tamamen çıkarılması önemlidir.
- **Sayfalama kullanmak:** Uzun listeler (örneğin haber akışı, mesaj listeleri vb.) mutlaka sayfalama ile tasarlanmalıdır. Örnek: `limit: 20` ve *cursor* tabanlı sayfalama.
- **Hafif ve ağır payload'ları ayırmak:** Liste ekranlarında hafif alanlara, detay ekranlarında daha ağır ve kapsamlı alanlara yer verilmesi, performans açısından faydalıdır.
- **Tek istekle birden fazla kaynağa erişmek:** Aynı ekranda normalde farklı REST endpoint'lerinden elde edilecek verileri tek bir GraphQL sorgusunda toplamak, özellikle mobil ortamlarda toplam gecikmenin azaltılmasına anlamlı katkı sağlar.

Mobil geliştirme perspektifinden bakıldığında GraphQL:

- Veri boyutunun kontrol edilmesini kolaylaştırmakta (örneğin 1 MB yerine 100 KB seviyeleri),
- Fazla sayıda REST endpoint bağımlılığını azaltarak **daha düşük gecikme** ve **daha az ağ trafiği** sağlamaya yardımcı olmakta,
- API'nin zaman içerisinde yeni sürümler tanımlamaya gerek kalmaksızın evrimleşmesine olanak tanımaktadır.

GraphQL ile Tüm Veriyi Tek Seferde Çekmek

Mobil uygulama mimarisinde zaman zaman, *“tüm veriyi tek bir GraphQL sorgusu ile çekip daha sonra tamamen yerel (local) bellekten kullanmak”* şeklinde bir yaklaşım gündeme gelebilmektedir. Bu bölümde, söz konusu yaklaşımın hangi durumlarda uygun olabileceği ve hangi durumlarda sakıncalı sonuçlar doğurabileceği ele alınmaktadır.

Tek Seferde Tüm Veriyi Çekme Yaklaşımı

Bu yaklaşımda temel fikir, aşağıdakine benzer kapsamlı bir sorgu ile uygulamanın ihtiyaç duyacağı hemen hemen tüm verilerin tek seferde elde edilmesidir:

```
query InitApp {  
  me { ...birçok alan }  
  allCategories { ...tüm alanlar }  
  allSpots { ...tüm alanlar }  
  allQuestions { ...tüm alanlar }  
  allFestivals { ...tüm alanlar }  
  # vb.  
}
```

Elde edilen yanıt, genellikle tek bir büyük nesne (örneğin bir global store veya yerel ön bellek) içerisinde saklanmakta; uygulama ekranları açıldıkça yeni ağ isteği yapılmaksızın bu nesneden okunmaktadır.

Teorik olarak bu yaklaşım, ilk açılışta nispeten daha fazla bekleme süresi karşılığında, sonrasında hızlı ve tamamen yerel okuma imkânı sunmayı amaçlamaktadır. Ancak uygulamada bu model her veri türü için uygun değildir.

Uygun Olduğu Durumlar

Aşağıdaki veri türleri için tüm veriyi tek seferde çekmek çoğu zaman anlamlı ve etkilidir:

- **Küçük ve statik referans veriler:** Cinsiyet seçenekleri, dil listeleri, sabit kategori türleri, ikon eşlemeleri gibi boyutu düşük ve nadiren değişen sözlük (dictionary) verileri.
- **Konfigürasyon ve özellik bayrakları (feature flags):** Örneğin, belirli bir özelliğin aktif olup olmadığı, minimum desteklenen uygulama sürümü gibi uygulama genelini ilgilendiren ayarlar.
- **Sınırlı boyutta sabit içerik katalogları:** Örneğin 30–100 öge aralığında kalan, içeriği sık değişmeyen küçük kataloglar.

Bu tür veriler, uygulamanın ömrü boyunca sıkça başvurulana, ancak boyutu görece küçük olan veri kümeleri olduğundan, tek seferde alınarak yerel bellekte veya kalıcı bir depoda (örneğin bir key-value store) tutulmaları performans açısından avantaj sağlayabilir.

Sakıncalı Olduğu Durumlar

Aşağıdaki veri tipleri için tüm veriyi tek bir sorgu ile çekmeye çalışmak genellikle önerilmemektedir:

- **Büyüyen ve kullanıcı tarafından üretilen veriler:** Sohbet geçmişleri, kullanıcıların oluşturduğu içerikler, büyük favori listeleri veya zamanla artan kayıtlar gibi veri kümeleri, kullanıcı sayısı ve kullanım süresi arttıkça hızla büyüyebilir. Tüm bu veriyi tek seferde çekmek, başlangıçta kabul edilebilir görünse bile zaman içinde büyük boyutlara ulaşabilir.
- **Sayfalama gerektiren uzun listeler:** Haber akışı, keşfet sayfaları, geniş festival veya mekân listeleri gibi içerikler için sayfalama (*pagination*) kaçınılmazdır. Tüm listeyi tek seferde istemek, hem ağ hem de bellek açısından verimsizdir; kullanıcı çoğu zaman bu içeriğin yalnızca küçük bir bölümünü görmektedir.
- **Sık güncellenen veya gerçek zamanlıya yakın veriler:** Kampanyalar, bildirimler, canlı skorlar gibi verilerin tek bir başlangıç sorgusu ile indirilip uzun süre güncellenmemesi, kullanıcının eski (geçerliliğini yitirmiş) verilerle karşılaşmasına neden olabilir. Her açılıшта aynı büyük sorguyu tekrar çalıştırmak ise ağ yükünü artırır.
- **Bellek ve performans etkileri:** Tek bir büyük sorgu sonucunda:
 - Büyük miktarda JSON verisinin ayrıştırılması (parse) ek CPU maliyeti oluşturur,
 - Uygulamanın JavaScript bellek alanında çok büyük nesneler tutulmasına neden olur,
 - Bu durum çöp toplama (garbage collection) sıklığını artırarak arayüzde takılmalara (jank) yol açabilir.

GraphQL'in Amacı Açısından Değerlendirme

GraphQL'in temel kazanımı, istemciye *yalnızca ihtiyaç duyduğu alanları ve ilişkileri* seçme imkânı vermesidir. Tüm veriyi tek bir sorgu ile istemek, bu ince taneli kontrol mekanizmasını kullanmamak anlamına gelir. Bu durumda, GraphQL kullanıyor olmanın getirdiği avantajlar önemli ölçüde azalmaktadır.

Dolayısıyla, *her şeyi tek bir sorguyla* almak yerine, GraphQL sorgularını **ekran bazlı, ihtiyaç odaklı** ve **boyut olarak sınırlı** şekilde tasarlamak, hem teknolojinin ruhuna hem de mobil performans gereksinimlerine daha uygundur.

Önerilen Hibrit Yaklaşım

Pratikte en sağlıklı model, *hibrit* bir stratejidir:

- **Başlangıç (bootstrap) sorgusu:** Uygulama ilk açıldığında veya oturum açma sonrasında, temel kullanıcı bilgileri, uygulama konfigürasyonu ve küçük referans veri setlerini içeren sınırlı bir InitApp sorgusu çalıştırılabilir. Örneğin:

```
query InitApp {
  me {
    id
    gender
    language
    premiumStatus
  }
  appConfig {
    minVersion
    features {
      name
      enabled
    }
  }
  staticDictionaries {
    genders { code label }
    relationshipGoals { code label }
  }
}
```

Bu sorgunun yanıtı, uygulamanın durum yönetiminde veya yerel saklama katmanında tutulabilir ve ekranlar bu bilgilerden yararlanabilir.

- **Ekran bazlı sorgular:** Her ekran, yalnızca kendi ihtiyaç duyduğu veri alanlarını içeren, uygun ölçüde küçük ve genellikle sayfalı (*paginated*) sorgular ile desteklenmelidir. Örneğin:
 - Ana ekran için `featuredSpots(limit: 5)` ve `todayPromotions`,
 - Festival listesi için `festivals(page: 1, limit: 20)`,
 - Soru listesi için `questionsByCategory(categoryId, limit: 50)`.

Bu sorguların sonuçları da uygun görüldüğünde yerel önbellekte saklanabilir ve arka planda yenileme (*refresh*) mekanizmalarıyla güncel tutulabilir.

Temel Tip Kavramları

Bu bölümde GraphQL tip sisteminin temel kavramları açıklanmakta ve her kavram için kısa, pratik örnekler verilmektedir. Örnekler, mobil uygulama geliştirme bağlamına yakın olacak şekilde seçilmiştir.

Schema (Şema) Nedir?

Tanım: Schema, bir GraphQL API'sinden hangi verilerin istenebileceğini, hangi tiplerin ve alanların mevcut olduğunu ve bu alanların tiplerini tanımlayan yapıdır. Kısaca, API'nin *sözlüğü* olarak düşünülebilir. Örneğin bir ilişki uygulamasında basit bir şema parçası şu şekilde olabilir:

```
type User {  
  id: ID!  
  name: String!  
  gender: String  
}
```

```
type Query {  
  me: User  
}
```

Bu tanıma göre:

- Sistemde bir User tipi vardır.
- User tipinin alanları:
 - id: ID! (her zaman dolu, null olamaz),
 - name: String! (her zaman dolu),
 - gender: String (boş olabilir).
- Query kök tipinde me adında bir alan vardır ve bu alan bir User döndürür.

Mobil geliştirici açısından, şema sayesinde:

- Hangi alanların sorgulanabileceği,
- Hangi alanların zorunlu, hangilerinin isteğe bağlı olduğu,
- Bir alanın tipinin ne olduğu

gibi bilgiler otomatik olarak öğrenilebilir.

Object Type Nedir?

Tanım: Object Type, belirli bir nesne türünü ve bu nesneye ait alanları tanımlayan tiptir. Nesne yönelimli programlamadaki `class User { name; age; }` yapısına benzetilebilir. GraphQL’de `type User { ... }` biçiminde ifade edilir.

Örneğin romantik mekan kartlarını temsil eden bir tip:

```
type Spot {  
  id: ID!  
  title: String!  
  city: String!  
  imageUrl: String  
}
```

Bu tanım:

- Spot adında bir Object tipin olduğunu,
- id, title, city alanlarının zorunlu olduğunu,
- imageUrl alanının ise isteğe bağlı (null olabilir) olduğunu

ifade eder.

Basit bir sorgu örneği:

```
{  
  romanticSpots(limit: 2) {  
    id  
    title  
    city  
  }  
}
```

Olası JSON yanıtı:

```
{  
  "data": {  
    "romanticSpots": [  
      { "id": "1", "title": "Kaputaş Plajı", "city": "Antalya" },  
      { "id": "2", "title": "Pokut Yaylası", "city": "Rize" }  
    ]  
  }  
}
```

Scalar Type Nedir?

Tanım: Scalar tipler, bir sorgunun uç (leaf) noktalarındaki basit değerleri temsil eder. Altında başka alan içermezler. Örnekler: Int, Float, String, Boolean, ID.

Örneğin:

```
type User {  
  name: String!  
  age: Int  
  isPremium: Boolean!  
}
```

Burada:

- name: String! (zorunlu),
- age: Int (isteğe bağlı),
- isPremium: Boolean! (zorunlu)

alanlarının her biri birer Scalar tiptir.

Sorgu:

```
{  
  me {  
    name  
    age  
    isPremium  
  }  
}
```

Örnek yanıt:

```
{  
  "data": {  
    "me": {  
      "name": "Ömer",  
      "age": 30,  
      "isPremium": true  
    }  
  }  
}
```

Scalar değerler tek parça olduğu için, bir scalar alanın altına ek alt alan seçimi yapılamaz (örneğin `me.name.something` gibi bir sorgu geçersizdir).

Non-Null (!) Nedir?

Tanım: GraphQL’de tipler varsayılan olarak *nullable* kabul edilir; yani bir alan `null` döndürebilir. Bir tipin sonuna `!` eklendiğinde, o alanın veya argümanın `null` olmaması gerektiği ifade edilir. Sunucu `null` döndürmeye çalışırsa, GraphQL çalıştırma aşamasında hata üretir.

Örneğin:

```
type User {  
  id: ID!           # null olamaz  
  name: String!     # null olamaz  
  bio: String       # null olabilir  
}
```

Bu tanımda:

- `id` ve `name` alanlarının her durumda bir değer döndürmesi gerekir,
- `bio` alanı boş bırakılabilir (örneğin yeni kayıt olan bir kullanıcı henüz biyografi girmemiş olabilir).

Mobil arayüz tasarımında bu bilgi, zorunlu alanlar için boş değer kontrolü derdini azaltır ve isteğe bağlı alanlar için uygun *fallback* metinlerin gösterilmesini mümkün kılar.

Liste ([]) Nedir?

Tanım: Köşeli parantezler ([]), ilgili alanın bir *dizi* (liste) döndürdüğünü veya bir argümanın dizi beklediğini ifade eder. Örneğin: [String], [Episode!], [Spot!]!.

Örnek bir tanım:

```
type User {  
    id: ID!  
    favoriteSpotIds: [ID!]!  
}
```

Burada favoriteSpotIds alanı için:

- Alan her zaman bir liste döndürür (alanın kendisi null olamaz),
- Liste boş olabilir ([]),
- Listenin içindeki her bir öğe ID! tipindedir ve null olamaz.

Geçerli ve geçersiz örnekler:

```
favoriteSpotIds: null           # geçersiz (Non-Null liste)  
favoriteSpotIds: []            # geçerli (hiç favorisi yok)  
favoriteSpotIds: ["1", "2"]    # geçerli  
favoriteSpotIds: ["1", null]   # geçersiz (ID! öğeleri null olamaz)
```

Bu davranış, mobil tarafta her zaman bir dizi bekleyebileceğiniz anlamına gelir; böylece listelemelerde (örneğin FlatList) ek null kontrollerine daha az ihtiyaç duyarsınız.

Argüman (Field Parametresi) Nedir?

Tanım: GraphQL’de her alan, bir fonksiyon gibi davranarak, sıfır veya daha fazla sayıda isimlendirilmiş argüman alabilir. Bu argümanlar, sorgu sırasında alanın nasıl davranacağını belirler.

Örnek:

```
type Query {  
  spot(id: ID!): Spot  
  searchSpots(city: String, limit: Int = 10): [Spot!]!  
}
```

Bu tanımda:

- `spot(id: ID!)` alanı, `id` isimli zorunlu bir argüman alır; `id` için `null` değer verilemez.
- `searchSpots(city, limit = 10)` alanında:
 - `city` isteğe bağlıdır,
 - `limit` verilmezse varsayılan olarak 10 kabul edilir.

Sorgu örneği:

```
{  
  searchSpots(city: "İstanbul", limit: 5) {  
    id  
    title  
  }  
}
```

Root Tipler: **Query**, **Mutation**, **Subscription**

Tanım: Root tipler, GraphQL şemasının giriş noktalarını temsil eder:

- **Query:** Okuma (sorgulama) operasyonlarının kök tipi,
- **Mutation:** Yazma/değiştirme operasyonlarının kök tipi,
- **Subscription:** Gerçek zamanlı veri akışı (örneğin WebSocket) için kök tip.

Örneğin:

```
type Query {  
  me: User  
  romanticSpots(limit: Int!): [Spot!]!  
}
```

```
type Mutation {  
  addFavorite(spotId: ID!): User  
}
```

Mobil geliştirici açısından:

- **Query** alanları liste ve detay ekranlarını beslemek için kullanılır,
- **Mutation** alanları, “favoriye ekle”, “beğen”, “yorum gönder” gibi işlem butonlarının arkasındaki operasyonlarda kullanılır,
- **Subscription** ise canlı sohbet, canlı skor gibi gerçek zamanlı senaryolarda devreye girer (temel kullanım için zorunlu değildir).

Interface Nedir?

Tanım: Interface tipler, soyut tiplerdir ve kendisini implemente eden Nesne tiplerinin sahip olması gereken ortak alanları tanımlar. Böylece, farklı tipler arasında ortak bir *kontrat* oluşturulmuş olur.

Örneğin:

```
interface Content {
    id: ID!
    title: String!
}

type Spot implements Content {
    id: ID!
    title: String!
    city: String!
}

type Festival implements Content {
    id: ID!
    title: String!
    country: String!
}
```

Burada:

- Content interface'i, her içeriğin en az id ve title alanlarına sahip olacağını belirtir.
- Spot ve Festival tipleri, bu interface'i implemente ederek bu ortak alanları sağlamak zorundadır.

Eğer bir alan `homeFeed: [Content!]!` şeklinde tanımlanmışsa:

```
{
  homeFeed {
    id
    title
  }
}
```

sorgusu, `homeFeed` içerisinde hem `Spot` hem de `Festival` nesneleri bulunsa dahi, her biri için id ve title alanlarına güvenle erişebildiğiniz anlamına gelir.

Tipe özgü alanlara erişmek için ise *inline fragment* kullanılır:

```
{  
  homeFeed {  
    id  
    title  
    ... on Spot {  
      city  
    }  
    ... on Festival {  
      country  
    }  
  }  
}
```

Bu sayede, ortak alanlar interface üzerinden, tipe özel alanlar ise ilgili inline fragmentler üzerinden sorgulanabilir.

Union Tipler

Union tipler, bir alanın birden fazla farklı Nesne (Object) tipinden herhangi birini döndürebileceğini ifade eden soyut tiplerdir. Union tipler, bir Interface tipinden farklı olarak, üye tipler arasında ortak alanlar tanımlamaz; yalnızca bu alanın hangi Nesne tiplerinden biri olabileceğini belirtir.

Örneğin, bir arama (search) işlevinin, hem mekan (Spot), hem festival (Festival), hem de kullanıcı (User) döndürebildiği bir senaryoda, aşağıdaki şema tanımı kullanılabilir:

```
type Spot {
  id: ID!
  title: String!
  city: String!
}

type Festival {
  id: ID!
  title: String!
  city: String!
}

type User {
  id: ID!
  name: String!
}

union SearchResult = Spot | Festival | User

type Query {
  search(text: String!): [SearchResult!]!
}
```

Bu tanım, search alanının döndürdüğü her bir elemanın Spot, Festival veya User tiplerinden biri olabileceğini ifade eder. İstemci tarafında, hangi tipte bir nesneyle karşılaşıldığını anlamak ve tipe özgü alanları sorgulamak için *inline fragment* ve `__typename` meta-alanı kullanılır:

```
{
  search(text: "istanbul") {
    __typename
    ... on Spot {
      title
      city
    }
  }
}
```

```

    }
    ... on Festival {
      title
      city
    }
    ... on User {
      name
    }
  }
}

```

Olası bir yanıt örneği:

```

{
  "data": {
    "search": [
      { "__typename": "Spot",
        "title": "Galata Kulesi",
        "city": "İstanbul"
      },
      { "__typename": "Festival",
        "title": "İstanbul Caz Festivali",
        "city": "İstanbul"
      },
      { "__typename": "User",
        "name": "İstanbulKızı"
      }
    ]
  }
}

```

Bu yapı, farklı tipte sonuçların tek bir liste altında toplanabilmesini sağlar; istemci, `__typename` ve inline fragmentler aracılığıyla her bir nesnenin gerçek tipine göre alanlara erişebilir.

Input Object Tipler

Input Object tipler, alan veya directif argümanlarının daha karmaşık, nesne benzeri yapılar halinde gruplanmasını sağlayan tiplerdir. Özellikle, çok sayıda parametre alan sorgu filtreleri veya mutation argümanları için faydalıdır.

Örneğin, romantik mekanları belirli filtrelere göre aramak istediğimizi varsayalım. Bireysel argümanlar yerine, tek bir filtre nesnesi tanımlanabilir:

```
input SpotFilterInput {
  city: String
  minRating: Float
  maxBudget: Int
}

type Spot {
  id: ID!
  title: String!
  city: String!
  rating: Float
}

type Query {
  searchSpots(filter: SpotFilterInput): [Spot!]!
}
```

Bu tanımda `SpotFilterInput`, `searchSpots` alanının `filter` argümanı için kullanılan bir Input Object tipidir. İstemci tarafında bu argüman şu şekilde kullanılabilir:

```
{
  searchSpots(filter: {
    city: "İzmir"
    minRating: 4.5
  }) {
    id
    title
    city
    rating
  }
}
```

Bu yaklaşımın avantajları şunlardır:

- İlgili argümanlar tek bir yapı altında mantıksal olarak gruplanır.

- Argüman sayısı arttığında şema tanımı daha derli toplu kalır.
- İstemci tarafında, tek bir nesne (`filter`) hazırlanarak sorguya parametre olarak verilebilir.

Input Object tipler yalnızca giriş (*input*) bağlamında kullanılır; çıktı (*output*) olarak bir alanın dönüş tipi şeklinde tanımlanamazlar ve alan düzeyinde argüman içeremezler.

Directifler

Directifler, @ karakteri ile başlayan ve şema veya sorgu parçalarının nasıl yorumlanacağını etkileyen açıklayıcı yapılardır. Tip sistemi directifleri, şemadaki tiplerin, alanların ve argümanların doğrulanma veya yürütülme biçimini değiştirmek için kullanılabilir.

GraphQL spesifikasyonu, bazı yerleşik directifleri tanımlar. Bunlardan biri de, artık kullanılması önerilmeyen alanların işaretlenmesi için kullanılan @deprecated directifidir:

```
type User {
  fullName: String
  name: String @deprecated(reason: "Use `fullName`.")
}
```

Bu örnekte:

- name alanı hâlen şemada mevcuttur,
- Ancak @deprecated ile işaretlendiği için, yeni istemcilerin fullName alanını kullanması teşvik edilir,
- GraphQL araçları (örneğin geliştirme ortamı eklentileri) name alanı sorgulandığında uyarı gösterebilir.

@deprecated directifinin olası tanımı SDL ile şu şekilde ifade edilebilir:

```
directive @deprecated(
  reason: String = "No longer supported"
) on FIELD_DEFINITION | ENUM_VALUE
```

Bu tanımda:

- reason argümanı, varsayılan değeri olan opsiyonel bir Stringtir,
- Directifin uygulanabileceği yerler, `FIELD_DEFINITION` ve `ENUM_VALUE`.

Benzer biçimde, uygulamaya özgü özel (custom) directifler de tanımlanabilir; bu directiflerin nasıl yorumlanacağı, kullanılan GraphQL kütüphanesinin ve sunucu implementasyonunun sorumluluğundadır.

Dokümantasyon: Açıklamalar ve Yorumlar

GraphQL, şemadaki tipler, alanlar ve argümanlar için dokümantasyon eklenmesini teşvik eder. Bu dokümantasyon iki ana mekanizma ile sağlanır: *açıklamalar* (descriptions) ve *yorumlar* (comments).

Açıklamalar (Descriptions)

Açıklamalar, Markdown sözdizimi destekleyen metinlerdir ve şemadaki tiplerin, alanların ve argümanların üzerinde yer alır. SDL'de çoğunlukla üç tırnak işareti ile tanımlanırlar:

```
"""
A character from the Star Wars universe
"""
type Character {
  "The name of the character."
  name: String!
}
```

Benzer şekilde, bir Enum tipi için:

```
"""
The episodes in the Star Wars trilogy
"""
enum Episode {
  "Star Wars Episode IV: A New Hope, released in 1977."
  NEWHOPE
  "Star Wars Episode V: The Empire Strikes Back, released in 1980."
  EMPIRE
  "Star Wars Episode VI: Return of the Jedi, released in 1983."
  JEDI
}
```

Bu açıklamalar:

- Şemayı insanlar için daha anlaşılır ve kendini ifade eden hâle getirir,
- Introspection sorguları aracılığıyla istemci tarafında da erişilebilir,
- GraphiQL ve benzeri geliştirme araçlarında, otomatik tamamlama ve yardım metinleri olarak görüntülenir.

Yorumlar (Comments)

Bazen şemaya, doğrudan tip veya alan açıklaması olmayan, yalnızca geliştiriciye yönelik notlar eklemek gerekebilir. Bu tür notlar, yorum olarak eklenir ve GraphQL ayrıştırıcısı tarafından yok

sayılır. SDL'de tek satırlık yorumlar # karakteri ile başlar:

```
# This line is treated like whitespace and ignored by GraphQL
type Character {
  name: String!
}
```

Yorumlar, yalnızca şemada değil, istemci sorgularında da kullanılabilir:

```
{
  hero {
    name
    # Queries can have comments!
    friends {
      name
    }
  }
}
```

Bu yorumlar, sunucu tarafından işlenmez; sorgunun okunabilirliğini artırmak amacıyla istemci tarafında kullanılır.

Özetle:

- **Union** tipler, tek bir alanın birden fazla farklı Nesne tipinden birini döndürebilmesini sağlar ve bu tipler arasında ortak alan tanımlı yapmaz.
- **Input Object** tipler, özellikle filtreler ve mutation argümanları gibi karmaşık giriş verilerini tek bir nesne altında toplamak için kullanılır.
- **Directifier**, tipler, alanlar ve argümanların doğrulanma ve yürütülme biçimini etkilemek için şema veya sorgu üzerinde ek meta bilgi sağlar; @deprecated bunun yaygın bir örneğidir.
- **Açıklamalar** ve **yorumlar**, GraphQL şemasının dokümantasyonunu zenginleştirir; açıklamalar istemci tarafından da görülebilen, yorumlar ise ayrıştırıcı tarafından yok sayılan notlardır.

Sorgu Seviyesinde Temel-İleri Kavramlar

Bu bölümde, GraphQL sorgularında sık kullanılan ve pratikte büyük fayda sağlayan kapsamlı kavramlar ele alınmaktadır.

Query Nedir, Nereden Başlar?

GraphQL’de veri okumak için yapılan operasyona **query** denir.

Her query, şemadaki *root* Query tipinden başlar:

```
type Query {  
  hero: Character  
}
```

Örnek sorgu:

```
{  
  hero {  
    name  
  }  
}
```

Olası cevap:

```
{  
  "data": {  
    "hero": {  
      "name": "R2-D2"  
    }  
  }  
}
```

Önemli noktalar:

- Yanıt her zaman en üst seviyede `data` alanı içinde döner.
- Hata varsa ayrıca `errors` alanı bulunur.
- Sorgunun yapısı ile dönen verinin yapısı **birebir aynıdır**.

Field ve Alt Field Mantığı

GraphQL’de sorgu yazarken temel olarak şu yapılır: “şu field’ı ver, bunun altındaki şu field’ları da ver”.

Örneğin, alt alanlarıyla birlikte bir sorgu:

```
{  
  hero {  
    name  
    friends {  
      name  
    }  
  }  
}
```

Burada:

- `hero` alanı bir `Character` döndürmektedir.
- `friends` alanı bir liste (dizi) döndürmektedir.
- Listedeki her bir `friend` için `name` alanı istenmektedir.

Notlar:

- Sorgu yazarken tekil veya liste için kullanılan sözdizimi aynıdır.
- Bir alanın tekil mi yoksa liste mi döndürdüğü, şemadaki tip tanımından anlaşılır (`Character` vs. `[Character!]`!).

Argümanlar (Field'e Parametre Geçmek)

Her field, bir fonksiyon gibi argüman alabilir:

```
type Query {  
  human(id: ID!): Human  
}
```

Örnek sorgu:

```
{  
  human(id: "1000") {  
    name  
    height  
  }  
}
```

Olası cevap:

```
{  
  "data": {  
    "human": {  
      "name": "Luke Skywalker",  
      "height": 1.72  
    }  
  }  
}
```

Argümanlar sadece üst seviyedeki alanlarda değil, alt alanlarda da kullanılabilir:

```
{  
  human(id: "1000") {  
    name  
    height(unit: FOOT)  
  }  
}
```

Burada `height(unit: FOOT)` ifadesi:

- Sunucu tarafında, metreyi feet'e çevirme mantığının tek bir yerde tanımlanmasını sağlar.
- Her istemcinin aynı dönüşümü kendi içinde ayrı ayrı yapmak zorunda kalmasını engeller.

Operation Type ve Operation Name

Şimdiye kadar çoğunlukla kısayol sözdizimi kullanılmıştır:

```
{
  hero {
    name
  }
}
```

Tam yazımı ise şöyledir:

```
query HeroNameAndFriends {
  hero {
    name
    friends {
      name
    }
  }
}
```

Burada:

- `query` = *operation type* (işlem türü: `query` / `mutation` / `subscription`),
- `HeroNameAndFriends` = *operation name* (fonksiyon ismi gibi düşünülebilir).

Neden önemlidir?

- Log'larda ve hata mesajlarında hangi sorgunun hata verdiğini anlamak kolaylaşır (örneğin: "HeroNameAndFriends query'si hata verdi").
- Aynı doküman içinde birden fazla operasyon tanımlanıyorsa, isim vermek zorunludur.

Alias (Field'e Takma Ad Vermek)

Sorun: Aynı field'i, farklı argümanlarla, tek bir sorgu içinde çağırmak istiyorsunuz.

Örnek:

```
query {  
  empireHero: hero(episode: EMPIRE) {  
    name  
  }  
  jediHero: hero(episode: JEDI) {  
    name  
  }  
}
```

Olası cevap:

```
{  
  "data": {  
    "empireHero": { "name": "Luke Skywalker" },  
    "jediHero": { "name": "R2-D2" }  
  }  
}
```

Burada:

- empireHero ve jediHero aynı field (hero) için kullanılan alias (takma ad) değerleridir.
- Alias kullanılmasa, aynı seviyede iki ayrı hero alanı tanımlanmaya çalışılacağı için çakışma ve hata oluşur.

Değişkenler (Variables) ile Sorguyu Dinamik Yapmak

Query içindeki argümanları doğrudan string içinde sabitlemek yerine, GraphQL'de değişkenler (*variables*) kullanılır.

1) Query'de değişken tanımlama:

```
query HeroNameAndFriends($episode: Episode) {  
  hero(episode: $episode) {  
    name  
    friends {  
      name  
    }  
  }  
}
```

2) Değerleri ayrı JSON olarak gönderme:

```
{  
  "episode": "JEDI"  
}
```

Olası cevap:

```
{  
  "data": {  
    "hero": {  
      "name": "R2-D2",  
      "friends": [ ... ]  
    }  
  }  
}
```

Bu yaklaşımın başlıca avantajları:

- Query string'ini çalışma zamanında string birleştirerek üretme ihtiyacı ortadan kalkar.
- Güvenlidir; kullanıcı girdisi doğrudan query string'ine gömülmez.
- Query metni sabit kalır, sadece *variables* JSON içeriği değişir.

Variable tanımı ve kurallar Sorgu başındaki (*\$episode: Episode*) kısmı, değişken tanımını ifade eder.

- Değişken tipi **Scalar**, **Enum** veya **Input Object** olmalıdır.
- Eğer field argümanı Non-Null ise (örneğin *id: ID!*), buna karşılık gelen değişken de Non-Null olmalıdır (*\$id: ID!*).

Varsayılan değer (default value)

```
query HeroNameAndFriends($episode: Episode = JEDI) {  
  hero(episode: $episode) {  
    name  
  }  
}
```

Bu durumda:

- Değişken gönderilmezse JEDI değeri kullanılır.
- Değişkenler sözlüğünde bir değer sağlanırsa, bu değer varsayılanın yerini alır.

Argümanlar (Field Parametreleri)

GraphQL'de her alan, bir fonksiyon gibi düşünülebilir ve sıfır ya da daha fazla sayıda argüman alabilir. Bu argümanlar, alanın döndüreceği veriyi filtrelemek, sıralamak veya sınırlandırmak için kullanılır.

Örnek bir Query tanımı:

```
type Query {  
  spot(id: ID!): Spot  
  searchSpots(city: String, limit: Int = 10): [Spot!]!  
}
```

Bu tanıma göre:

- `spot(id: ID!): Spot` alanı, `id` isimli zorunlu bir argüman alır ve tek bir `Spot` nesnesi döndürür.
- `searchSpots(city: String, limit: Int = 10): [Spot!]!` alanı, isteğe bağlı `city` argümanını ve varsayılan değeri 10 olan `limit` argümanını alarak bir `Spot` listesi döndürür.

Örnek bir sorgu:

```
{  
  searchSpots(city: "İstanbul", limit: 5) {  
    id  
    title  
  }  
}
```

Bu yapı sayesinde, klasik REST yaklaşımında yalnızca endpoint ve query parametreleri düzeyinde tanımlanabilen filtreler, GraphQL'de her alan için ayrı ayrı tanımlanabilir hale gelmektedir.

İşlem Tipi ve İşlem Adı (Operation Type ve Name)

GraphQL sorgularında, yapılan işlemin türü ve isteğe bağlı bir işlem adı belirtilerek sorgular daha ifade gücü yüksek ve izlenebilir hale getirilebilir.

Temel işlem tipleri:

- `query`: Veri okuma operasyonları için,
- `mutation`: Veri değiştirme (yazma) operasyonları için,
- `subscription`: Gerçek zamanlı veri akışı için.

Basit sorgu kısaltması:

```
{
  hero {
    name
  }
}
```

Tam yazımı, işlem tipi ve adı ile birlikte:

```
query HeroNameAndFriends {
  hero {
    name
    friends {
      name
    }
  }
}
```

Burada:

- `query` anahtar sözcüğü, bunun bir okuma operasyonu olduğunu belirtir.
- `HeroNameAndFriends` işlemin adıdır ve hata ayıklama, loglama gibi durumlarda hangi sorgunun çalıştığını anlamayı kolaylaştırır.

Bir GraphQL dokümanı içinde birden fazla operasyon tanımlanıyorsa işlem adı zorunludur; tek bir operasyon olduğunda da isim vermek, bakım ve hata ayıklama açısından tavsiye edilir.

Alias (Takma Ad) Kullanımı

Bir GraphQL sorgusunda aynı alanı farklı argümanlarla birden fazla kez kullanmak istendiğinde, sonuçta aynı alan adları çıkarılır. Bu durumda, *alias* kullanılarak her bir alan sonucuna farklı bir isim verilebilir.

Örnek:

```
query {  
  istanbulSpots: searchSpots(city: "İstanbul", limit: 3) {  
    title  
  }  
  izmirSpots: searchSpots(city: "İzmir", limit: 3) {  
    title  
  }  
}
```

Bu sorgunun olası yanıtı:

```
{  
  "data": {  
    "istanbulSpots": [  
      { "title": "Galata Köprüsü" },  
      { "title": "Moda Sahili" },  
      { "title": "Pierre Loti" }  
    ],  
    "izmirSpots": [  
      { "title": "Kordon" },  
      { "title": "Alsancak Sahili" },  
      { "title": "Seferihisar" }  
    ]  
  }  
}
```

Burada:

- Gerçek alan adı her iki durumda da `searchSpots`'tur.
- `istanbulSpots` ve `izmirSpots`, sonuç JSON'unda kullanılacak takma adlardır (alias).

Alias sayesinde, aynı alanı farklı argümanlarla tek bir sorgu içinde birden çok kez kullanılabilir ve sonuç verisi açık, anlamlı anahtarlar altında toplanabilir.

Değişkenler (Variables)

Gerçek uygulamalarda, sorgu içindeki argüman değerleri çoğunlukla dinamik olarak kullanıcıdan, route parametrelerinden veya form alanlarından gelir. Bu değerleri doğrudan sorgu metnine gömmek yerine, GraphQL'de *variables* mekanizması kullanılır.

Değişken Tanımı ve Kullanımı

Bir sorguda değişken kullanmak için üç adım izlenir:

1. Sorgu başında değişkenleri, isimleri ve tipleriyle birlikte tanımlamak,
2. Alan argümanlarında sabit değerler yerine bu değişkenleri kullanmak,
3. Değişken değerlerini, sorgudan ayrı bir JSON nesnesi olarak sunucuya göndermek.

Örnek sorgu:

```
query HeroNameAndFriends($episode: Episode) {  
  hero(episode: $episode) {  
    name  
    friends {  
      name  
    }  
  }  
}
```

Bu sorgu, aşağıdaki değişkenler ile birlikte gönderilebilir:

```
{  
  "episode": "JEDI"  
}
```

Burada:

- `$episode: Episode`, `$episode` isimli bir değişkenin `Episode` tipinde olduğunu belirtir.
- `hero(episode: $episode)` ifadesi, alan argümanının değerini bu değişkenden alır.
- Değişken değerleri, sorgudan bağımsız olarak JSON sözlüğü şeklinde iletilir.

Bu yaklaşım:

- Sorgu metninin sabit kalmasını,
- Kullanıcı girdilerini güvenli biçimde ayrı bir yapıdaki değişkenler aracılığıyla iletmeyi,
- Kodun okunabilirliğini ve bakımını kolaylaştırmayı

sağlar.

Varsayılan Değerler

Değişkenlerin tip tanımında varsayılan değer de belirtilebilir:

```
query HeroNameAndFriends($episode: Episode = JEDI) {  
  hero(episode: $episode) {  
    name  
  }  
}
```

Bu durumda:

- Herhangi bir episode değişkeni gönderilmezse, varsayılan olarak JEDI değeri kullanılır.
- Eğer değişkenler sözlüğünde bir değer sağlanırsa, bu varsayılanın yerini alır.

Fragmentler

Fragmentler, GraphQL sorgularında tekrar eden alan kümelerini yeniden kullanılabilir birimler hâlinde tanımlamayı sağlayan yapılardır. Özellikle karmaşık sayfaların veya bileşenlerin birden fazla yerde aynı alan setlerini talep etmesi durumunda faydalıdır.

Tekrarlı bir sorgu örneği:

```
query {  
  leftHero: hero(episode: EMPIRE) {  
    name  
    appearsIn  
    friends {  
      name  
    }  
  }  
  rightHero: hero(episode: JEDI) {  
    name  
    appearsIn  
    friends {  
      name  
    }  
  }  
}
```

Bu sorguda name, appearsIn ve friends { name } alanlarının tamamı iki defa yazılmıştır. Aynı alan kümesini bir fragment olarak tanımlayarak bu tekrar giderilebilir:

```
fragment HeroBasicInfo on Character {  
  name  
  appearsIn  
  friends {  
    name  
  }  
}
```

Fragment tanımlandıktan sonra sorgu şu şekilde sadeleştirilebilir:

```
query {  
  leftHero: hero(episode: EMPIRE) {  
    ...HeroBasicInfo  
  }  
  rightHero: hero(episode: JEDI) {  
    ...HeroBasicInfo  
  }  
}
```

```
}  
}
```

Burada:

- `fragment HeroBasicInfo on Character { ... }` ifadesi, `Character` tipi için bir alan kümesi tanımlar.
- `...HeroBasicInfo` yazıldığı yerde, söz konusu `fragment`'te tanımlı olan tüm alanlar sorguya dahil edilir.

Bu yaklaşım:

- Sorgu tanımlarını daha okunabilir ve bakım yapılabilir hâle getirir,
- Aynı alan setinin birden fazla bileşen veya ekran tarafından kullanıldığı durumlarda tek bir merkezden güncellenmesine imkân tanır.

Sorgu İçinde Directif Kullanımı: @include ve @skip

Executable directifler, bir GraphQL sorgusunda belirli alanların koşullu olarak işlenmesini sağlar. Bu sayede, aynı sorgu hem “sade” hem de “detaylı” kullanım senaryolarını destekleyebilir.

GraphQL spesifikasyonuna göre her sunucunun desteklemesi gereken iki built-in directif vardır:

- @include(if: Boolean): Argümanı true ise ilgili alan sonucu içine dahil edilir.
- @skip(if: Boolean): Argümanı true ise ilgili alan atlanır.

Örneğin, bir kahramanın (hero) arkadaşlarını yalnızca belirli bir durumda göstermek istediğimizi varsayalım:

```
query Hero($episode: Episode, $withFriends: Boolean!) {  
  hero(episode: $episode) {  
    name  
    friends @include(if: $withFriends) {  
      name  
    }  
  }  
}
```

Bu sorgu, aşağıdaki değişkenlerle gönderilebilir:

```
{  
  "episode": "JEDI",  
  "withFriends": false  
}
```

Bu durumda yanıt:

```
{  
  "data": {  
    "hero": {  
      "name": "R2-D2"  
    }  
  }  
}
```

şeklinde olacaktır; friends alanı tamamen yok sayılır. Eğer "withFriends": true gönderilirse, yanıt içinde friends alanı da yer alacaktır.

Bu mekanizma sayesinde:

- Sorgu metnine string manipölasyonu ile alan ekleyip çıkarmaya gerek kalmadan koşullu alan seçimleri yapılabilir,
- Aynı sorgu hem hafif (özet) hem de detaylı veri ihtiyaçlarını karşılayabilir.

Mutations’a Giriş

Mutation Kavramı ve Query ile Farkı

GraphQL’de veri okuma işlemleri **query** ile, veri üzerinde değişiklik yapan (ekleme, güncelleme, silme vb.) işlemler ise **mutation** ile gerçekleştirilir.

- **Query**: Sunucudaki veriyi okur ve döner; veriyi değiştirmemelidir.
 - Örnek: Bir kullanıcının favori mekânlarını listelemek.
- **Mutation**: Sunucudaki veriyi değiştirir (yeni veri ekler, mevcut veriyi günceller veya siler).
 - Örnek: Bir mekânı favorilere eklemek, kullanıcı adını güncellemek, bir kaydı silmek.

Bu bağlamda, SQL analojisiyle:

- SELECT → Query,
- INSERT, UPDATE, DELETE → Mutation

şeklinde düşünülebilir.

Yeni Veri Ekleme: **addFavorite** Örneği

Bir kullanıcı arayüzünde “Favorilere ekle” butonuna basıldığında, genellikle arka planda veritabanına yeni bir kayıt eklenmesi beklenir. Bu işlem, GraphQL tarafında bir mutation alanı olarak tanımlanır.

Şema Tanımı

```
type Mutation {  
  addFavorite(spotId: ID!): Favorite  
}  
  
type Favorite {  
  id: ID!  
  userId: ID!  
  spotId: ID!  
}
```

Bu tanıma göre:

- `addFavorite(spotId: ID!): Favorite` alanı, `spotId` isimli zorunlu bir argüman alır.
- Geriye, yeni oluşturulan favori kaydını temsil eden bir `Favorite` nesnesi döndürür.

Mutation Çağrısı

```
mutation AddFavorite($spotId: ID!) {  
  addFavorite(spotId: $spotId) {  
    id  
    spotId  
  }  
}
```

Değişkenler (Variables)

```
{  
  "spotId": "pokut-001"  
}
```

Örnek Yanıt

```
{  
  "data": {  
    "addFavorite": {  
      "id": "fav-123",  
      "spotId": "pokut-001"  
    }  
  }  
}
```

Bu akışta:

- `mutation AddFavorite($spotId: ID!)` ifadesi, veri değiştiren bir operasyon tanımlar.
- `addFavorite(spotId: $spotId)` alanı, ilgili mekânı favorilere ekler.
- Seçim kümesinde (`{ id spotId }`) yanıt içinde hangi alanların döndürüleceği belirtilir.

Mevcut Veriyi Güncelleme: `updateUserName` Örneği

Bir profil ekranında kullanıcının adını değiştirmesi senaryosunda, veritabanındaki `users` tablosu güncellenecektir. Bu işlem de bir mutation ile ifade edilir.

Şema Tanımı

```
type Mutation {  
  updateUserName(name: String!): User  
}
```

```
type User {  
  id: ID!  
  name: String!  
}
```

Mutation Çağrısı

```
mutation UpdateUserName($name: String!) {  
  updateUserName(name: $name) {  
    id  
    name  
  }  
}
```

Değişkenler

```
{  
  "name": "Ömer Yavuz"  
}
```

Örnek Yanıt

```
{  
  "data": {  
    "updateUserName": {  
      "id": "user-324d27d7",  
      "name": "Ömer Yavuz"  
    }  
  }  
}
```

Bu örnekte:

- `updateUserName` mutation alanı, sunucu tarafında kullanıcı adını günceller.
- Yanıtta güncellenmiş `id` ve `name` alanları döndürülerek, istemcinin sunucudaki güncel durumu güvenle kullanması sağlanır.

Query ve Mutation'ın Yan Yana Karşılaştırılması

Query Örneği (Sadece Okuma)

```
query GetFavorites {  
  myFavorites {  
    spotId  
    title  
  }  
}
```

Bu sorgu, favori mekânların okunmasını sağlar; veriyi değiştirmez.

Mutation Örneği (Değiştirme)

```
mutation AddFavorite($spotId: ID!) {  
  addFavorite(spotId: $spotId) {  
    id  
    spotId  
  }  
}
```

Bu mutation, belirtilen mekânı favorilere ekleyerek veri üzerinde değişiklik yapar.

Dolayısıyla:

- query anahtar sözcüğü ile başlayan operasyonlar veri okuma odaklıdır.
- mutation anahtar sözcüğü ile başlayan operasyonlar veri yazma/değiştirme odaklıdır.

Subscriptions’a Giriş

Subscription Kavramı

GraphQL’de veri okuma işlemleri **query** ile, veri üzerinde değişiklik yapan işlemler (ekleme, güncelleme, silme vb.) **mutation** ile gerçekleştirilir. Buna ek olarak, istemcinin sunucudan **gerçek zamanlı (veya gerçek zamana yakın) güncellemeler** almasını sağlayan üçüncü bir operasyon türü daha vardır: **subscription**.

Subscription, tek seferlik bir istek–yanıt modelinden farklı olarak, uzun ömürlü bir bağlantı üzerinden sunucudan gelen olayları dinlemeye imkân tanır. Örneğin:

- Yeni bir yorum (review) oluşturulduğunda haberdar olmak,
- Canlı sohbet mesajlarını almak,
- Canlı skor veya metrik güncellemelerini izlemek.

Subscription Tipi ve Operasyon Tanımı

Subscription alanları, Query ve Mutation alanlarında olduğu gibi şema üzerinde tanımlanır, ancak Subscription kök operasyon tipi kullanılır:

```
type Subscription {  
  reviewCreated: Review  
}
```

İstemci tarafında bir subscription operasyonu, `subscription` anahtar kelimesi ile başlatılır:

```
subscription NewReviewCreated {  
  reviewCreated {  
    rating  
    commentary  
  }  
}
```

Bu örnekte:

- `NewReviewCreated` operasyon adı,
- `reviewCreated` ise, yeni bir `Review` nesnesi yayımlandığında tetiklenecek subscription alanıdır.
- Seçim kümesinde (`{ rating commentary }`) hangi alanların her güncellemede istemciye gönderileceği belirtilmektedir.

Pub/Sub Mantığı ve Taşıma Katmanı

Çoğu gerçek dünyadaki uygulamada, subscription alanlarının arka planında bir *pub/sub* (publish/subscribe) sistemi bulunur. Tipik akış şu şekildedir:

- Örneğin bir `createReview` mutation'ı çalıştığında, sistem bir `REVIEW_CREATED` mesajı yayımlar.
- `reviewCreated` subscription alanının resolver'ı bu olayı dinler.
- İlgili olaya abone olmuş istemcilere, yeni veriyi içeren mesaj gönderilir.

GraphQL spesifikasyonu, subscription'ların hangi taşıma protokolü ile gerçekleştirileceğini tanımlamaz; bu tercih sunucuya bırakılmıştır. Uygulamada çoğunlukla:

- **WebSocket** veya
- **Server-Sent Events (SSE)**

kullanılmaktadır. İstemci kütüphanesinin de seçilen protokolü desteklemesi gerekmektedir.

Tek Kök Alan Kuralı

Subscription dokümanları birden fazla operasyon içerebilir; ancak **her subscription operasyonunun tam olarak bir adet kök alanı (root field) olmalıdır**. Aşağıdaki örnek bu nedenle geçersizdir:

```
subscription {
  reviewCreated {
    rating
    commentary
  }
  humanFriendsUpdated {
    name
    friends {
      name
    }
  }
}
```

Buna karşılık, aynı dokümanda birden fazla *ayrı isimlendirilmiş* subscription operasyonu tanımlanabilir:

```
subscription NewReviewCreated {
  reviewCreated {
    rating
  }
}
```

```

        commentary
    }
}

subscription FriendListUpdated($id: ID!) {
  humanFriendsUpdated(id: $id) {
    name
    friends {
      name
    }
  }
}

```

Bu durumda:

- Dokümanda birden fazla operasyon bulunduğu için her birine isim verilmesi zorunludur.
- İstek gönderilirken hangi operasyonun çalıştırılacağı, operasyon adı ile belirtilir.

Mimarî Zorluklar ve Kullanım Senaryoları

Subscription operasyonları, query ve mutation'a göre daha karmaşık bir mimarî gerektirir:

- Sunucu, subscription süresince GraphQL dokümanını, değişkenleri ve bağlam bilgisini (context) canlı tutmak zorundadır.
- Yatay ölçekleme senaryolarında, her abonenin belirli bir sunucu instance'ına bağlanması ve bu ilişkinin yönetilmesi gerekir.
- İstemci tarafında:
 - Bağlantı koptuğunda otomatik yeniden abone olma,
 - İlk query sonuçları ile subscription güncellemeleri arasındaki olası yarış durumlarını (race condition) yönetme gibi ek mekanizmalar gerekebilir.

Bu nedenle, subscription'lar özellikle:

- Verinin sık ve artımlı (incremental) olarak değiştiği,
- İstemcinin bu değişiklikleri olabildiğince hızlı bir biçimde gösterme ihtiyacı duyduğu,

durumlar için uygundur (canlı sohbet, bildirim akışı, gerçek zamanlı gösterge panelleri, vb.). Daha seyrek güncellenen veriler için ise aşağıdaki yaklaşımlar daha basit ve genellikle yeterli olabilir:

- Periyodik polling (belirli aralıklarla tekrar sorgu çekmek),
- Kullanıcı etkileşimine bağlı yeniden sorgulama,
- Mobil push bildirimleri ile tetiklenen veri yenileme.

GraphQL Validation’a Giriş

Validation (Doğrulama) Kavramı

Bir GraphQL isteği sunucuya ulaştığında, iki temel aşamadan geçer:

1. **Parse** aşaması: İstek dokümanının söz dizimi (syntax) kontrol edilir.
2. **Validation** aşaması: Doküman, şemaya (schema) göre doğrulanır.

Validation aşamasında şu sorulara yanıt aranır:

- İstenen field’lar gerçekten ilgili tip üzerinde tanımlı mı?
- Selection set’ler tip kurallarına uygun mu (scalar vs. object ayrımı)?
- Fragment’ler doğru tipler üzerinde ve doğru biçimde kullanılıyor mu?

Bu sayede, hatalı sorgular *çalıştırılmadan* önce tespit edilir ve geliştiriciye anlamlı hata mesajları döndürülür.

Var Olmayan Field İstemek

Bir field, yalnızca tanımlı olduğu tip üzerinde sorgulanabilir. Örneğin, şemada:

```
type Character {  
  name: String!  
  appearsIn: [Episode!]!  
}
```

tanımı varsa ve favoriteSpaceship alanı Character tipi üzerinde tanımlı değilse, aşağıdaki sorgu geçersizdir:

```
# INVALID: favoriteSpaceship does not exist on Character  
query {  
  hero {  
    favoriteSpaceship  
  }  
}
```

Bu durumda sunucu, örneğe benzer biçimde aşağıdaki türde bir hata döndürür:

```
{  
  "errors": [  
    {  

```

```
      "message": "Cannot query field \"favoriteSpaceship\" on type \"Character\""  
    }  
  ]  
}
```

Selection Set ve Leaf Field Kuralları

GraphQL'de selection set'ler için iki temel kural vardır:

Obje Döndüren Field'lar için Alt Alan Seçilmesi Zorunluluğu

Eğer bir field, scalar veya enum dışındaki bir tip (örneğin bir Object veya Interface) döndürüyorsa, o field için mutlaka bir selection set belirtilmelidir.

Örnek şema:

```
type Query {  
  hero: Character  
}
```

Aşağıdaki sorgu geçersizdir:

```
# INVALID: hero is not a scalar, so fields are needed  
query {  
  hero  
}
```

Doğru kullanım:

```
query {  
  hero {  
    name  
    appearsIn  
  }  
}
```

Scalar/Enum Tipler için Alt Alan Seçilmemesi

Scalar veya enum tipler, leaf (yaprak) değerlerdir; alt alanları olamaz. Örneğin:

```
type Character {  
  name: String!  
}
```

Aşağıdaki sorgu geçersizdir:

```
# INVALID: name is a scalar, so fields are not permitted  
query {  
  hero {  
    name {  
      firstCharacterOfName  
    }  
  }  
}
```

Sunucu bu durumda, name alanının String! tipi nedeniyle seçim kümesi (subfield) alamaya-
cağını belirten bir hata döndürür.

Validation Hatası Durumunda Davranış

Validation aşamasında bir hata tespit edildiğinde:

- İlgili istek için **alan yürütmesi (field execution)** başlatılmaz.
- Veritabanına veya arka uç (backend) sistemlerine herhangi bir erişim yapılmadan, hata bir **istek hatası (request error)** olarak geri döndürülür.
- Yanıtta genellikle errors alanı bulunur; data alanı boş veya null olabilir.

Örnek bir hata yanıtı şu şekilde olabilir:

```
{  
  "errors": [  
    {  
      "message": "...",  
      "locations": [{ "line": 4, "column": 5 }]  
    }  
  ]  
}
```

GraphQL spesifikasyonu, tüm uyumlu implementasyonların gelen istekleri şemaya göre validate etmesini zorunlu kıldığından, bu tip kontroller için ek çalışma zamanı (logic) yazmaya gerek kalmaz.

GraphQL Yanıt Formatı (Response)

GraphQL sunucusu, bir isteği (*request*) doğrulayıp çalıştırdıktan (*execute*) sonra istemciye bir yanıt (*response*) döner. Bu yanıtın temel özelliği, **şekil olarak** istemcinin gönderdiği sorgunun yapısını yansıtmasıdır. Ancak yanıtta sadece veri değil, aynı zamanda oluşan hatalar ve isteğe özgü ek bilgiler de yer alabilir.

GraphQL spesifikasyonuna göre bir yanıt, en üst seviyede en fazla üç anahtar (*key*) içerebilir:

- data
- errors
- extensions

Bu üç anahtardan en az birinin data veya errors olması zorunludur; yani yanıt her zaman veri ya da hata (veya her ikisini birden) içermelidir.

data Alanı

Başarılı bir GraphQL isteğinde dönen asıl sonuç, data anahtarının altında yer alır. Bu alanın içindeki yapı, sorgunun seçtiği alanlarla birebir uyumludur.

Örnek sorgu:

```
query {  
  human(id: 1002) {  
    name  
    appearsIn  
    starships {  
      name  
    }  
  }  
}
```

Örnek yanıt:

```
{  
  "data": {  
    "human": {  
      "name": "Han Solo",  
      "appearsIn": [  
        "NEWHOPE",  
        "EMPIRE",  
        "JEDI"  
      ]  
    }  
  }  
}
```

```

    ],
    "starships": [
      { "name": "Millenium Falcon" },
      { "name": "Imperial shuttle" }
    ]
  }
}

```

Dikkat edilmesi gereken noktalar:

- data en üst seviyede tek bir anahtar olarak bulunur.
- data içindeki yapı, sorguda seçilen alanların hiyerarşisi ile tamamen uyumludur.
- Spesifikasyon belirli bir serileştirme formatı şart koşturamaz; pratikte yanıtlar neredeyse her zaman JSON formatında döndürülür.
- İletim protokolü de zorunlu değildir; genellikle:
 - Sorgular ve mutasyonlar için HTTP,
 - Abonelikler (*subscriptions*) için WebSocket veya Server-Sent Events kullanılmaktadır.

errors Alanı

errors alanı, isteğin işlenmesi sırasında ortaya çıkan hataları taşır. Hataların türüne bağlı olarak yanıtın sadece errors içermesi ya da hem data hem errors içeren *kısmi* (*partial*) bir yanıt olması mümkündür.

Genel hata türleri:

- **İstek hataları** (*request errors*): Söz dizimi veya şema doğrulama hataları.
- **Field hataları** (*field errors*): Çalışma sırasında, belirli bir alanın çözülmesi (*resolver*) aşamasında oluşan hatalar.
- **Ağ hataları** (*network errors*): TCP/SSL/timeout gibi GraphQL'e özgü olmayan taşıma katmanı problemleri (genellikle JSON yanıt bile alınmaz).

İstek Hataları (Request Errors)

İstek hataları genellikle istemcinin oluşturduğu belgede (*document*) bir sorun olduğunda ortaya çıkar:

- Söz dizimi hataları (örneğin yanlış anahtar kelime, eksik parantez),

- Şemada bulunmayan bir alanın sorgulanması,
- Değişken (*variable*) tipi ile beklenen argüman tipinin uyuşmaması.

Bu durumda hata, *doğrulama aşamasında* tespit edilir ve alanlar henüz çalıştırılmadan istek reddedilir. Yanıtta data alanı bulunmaz, yalnızca errors yer alır.

Örnek: Şemada olmayan bir alanın istenmesi:

```
query {
  starship(id: 3000) {
    width    # Starship tipinde böyle bir field yok
  }
}
```

Örnek yanıt:

```
{
  "errors": [
    {
      "message": "Cannot query field \"width\" on type \"Starship\". Did you mea",
      "locations": [
        { "line": 3, "column": 5 }
      ]
    }
  ]
}
```

Örnek: Yanlış tipte değişken kullanımı:

```
query HeroForEpisode($ep: String!) {
  hero(episode: $ep) {
    name
  }
}
```

Değişken:

```
{
  "ep": "JEDI"
}
```

Yanıt:

```
{
  "errors": [
    {
      "message": "Variable \"$sep\" of type \"String!\" used in position \
expecting type \"Episode\".",
      "locations": [
        { "line": 1, "column": 22 },
        { "line": 2, "column": 17 }
      ]
    }
  ]
}
```

Bu tür durumlarda, istek hiç yürütülmediği için data anahtarı yer almaz.

Field Hataları (Field Errors)

Field hataları, istek şema açısından geçerli olduğu halde, *çalışma zamanında* belirli bir alanın çözümü sırasında oluşur. Örneğin:

- Resolver fonksiyonunun doğrudan hata fırlatması,
- Non-Null (!) olarak tanımlanmış bir alan için null döndürülmesi.

Bu durumda GraphQL, diğer alanları mümkün olduğu kadar çözmeye devam eder ve sonuçta:

- data alanı altında *kısmi* veri,
- errors alanı altında hata detayları

birlikte dönebilir.

Örnek mutation:

```
mutation {
  firstShip: deleteStarship(id: "3001")
  secondShip: deleteStarship(id: "3010")
}
```

Varsayım:

- "3001" kimliğine sahip gemi mevcut ve başarıyla siliniyor.
- "3010" kimliğine sahip gemi bulunamıyor ve hata oluşturuluyor.

Yanıt:

```
{
  "errors": [
    {
      "message": "Starship not found",
      "locations": [
        { "line": 3, "column": 3 }
      ],
      "path": ["secondShip"]
    }
  ],
  "data": {
    "firstShip": "3001",
    "secondShip": null
  }
}
```

Burada:

- `data.firstShip` alanı, başarılı işlemi temsil etmektedir.
- `data.secondShip` alanı, hata nedeni ile `null` dönmüştür.
- `errors[*].path` ile hangi field'in hata verdiği açıkça belirtilmiştir ("`secondShip`").

Ağ Hataları (Network Errors)

Ağ hataları, GraphQL'den bağımsız olarak taşıma katmanında ortaya çıkan sorunlardır. Örneğin:

- Bağlantı zaman aşımı,
- SSL hatası,
- TCP bağlantısının kopması.

Bu tür hatalarda çoğu zaman sunucudan hiç geçerli GraphQL yanıtı (`data/errors` içeren JSON) alınamaz. Bu nedenle, bu hatalar genellikle GraphQL yanıt formatının dışında, istemci kütüphanesinin ağ hatası (*network error*) mekanizmaları üzerinden ele alınır.

extensions Alanı

`extensions` anahtarı, spesifikasyonda ayrılmış özel bir alandır ve GraphQL implementasyonlarının yanıtı ek meta bilgi eklemesi için kullanılır. Bu alan:

- Opsiyoneldir.
- Varsa mutlaka bir nesne (*object*) olmalıdır.
- İçeriği tamamen ilgili sunucu implementasyonuna bağlıdır.

Örnek kullanım alanları:

- Telemetri veya *trace id* bilgileri,
- Rate limit kullanımı (kalan istek sayısı vb.),
- Sadece geliştirme ortamına özgü hata/diagnostik bilgileri.

Örneğin:

```
{
  "data": { ... },
  "extensions": {
    "requestId": "abc-123",
    "rateLimit": {
      "limit": 1000,
      "remaining": 742
    }
  }
}
```

GraphQL'in Tercih Edilmesinin Uygun Olduğu Durumlar

Birden Fazla İstemci Türü Bulunan Sistemler

Aynı arka uç (backend) sistemine farklı türde istemciler (örneğin web uygulaması, mobil uygulama, gelecekte eklenebilecek diğer istemciler) bağlanıyorsa, bu istemcilerin veri ihtiyaçları genellikle birbirinden farklı olacaktır.

REST mimarisinde bu durum, bir kaynağa ilişkin çok sayıda farklı uç nokta (`/user`, `/user/details`, `/user/stats` vb.) tanımlanmasına yol açabilmektedir. GraphQL ise tek bir şema üzerinde, istemcinin yalnızca ihtiyaç duyduğu alanları seçebilmesine imkân tanıyarak bu çeşitliliği daha derli toplu biçimde yönetmeyi sağlar.

Aynı Kaynaktan Farklı Alan Kombinasyonlarının Talep Edilmesi

Bir alan modeli (örneğin `User`) çok sayıda niteliğe sahipse ve farklı ekranlar bu modelden farklı alt kümelerde alanlar talep ediyorsa, GraphQL önemli avantaj sunar.

REST yaklaşımında:

- ya her istemci, gereğinden fazla veri içeren genel bir uç noktayı çağırmak zorunda kalır (overfetching),
- ya da her varyasyon için ayrı uç noktalar tasarlanır.

GraphQL'de ise aynı sorgu türü üzerinde farklı seçim kümeleri (*selection set*) tanımlanarak, her ekranın yalnızca ihtiyaç duyduğu alanları çekmesi mümkündür. Böylece istemci tarafında aşırı veri taşınması veya uç nokta çoğalması engellenmiş olur.

Mobil ve Düşük Bant Genişlikli Ortamlarda İstek Sayısının Azaltılması

Özellikle mobil istemcilerde ve kısıtlı ağ koşullarında, sunucuya yapılan istek sayısının azaltılması kullanıcı deneyimi açısından kritik önem taşır. REST tabanlı tasarımlarda, bir ekranın hazırlanması için çoğu zaman birden fazla uç noktaya ardışık istek yapılması gerekebilir.

GraphQL, tek bir uç nokta (`/graphql`) üzerinden çoklu veri ihtiyaçlarını tek istek ile karşılamaya imkân tanıyarak ağ gecikmesini ve toplam istek sayısını azaltır. Bu durum, özellikle mobil ağların kararsız veya yavaş olduğu senaryolarda anlamlı bir avantaj sağlar.

Farklı Veri Kaynaklarının Birleştirilmesi Gereken Senaryolar

Bazı sistemlerde veriler:

- ilişkisel veritabanı (ör. PostgreSQL),
- ödeme sağlayıcıları (ör. Google Play, Stripe),
- diğer REST servisleri veya önbellek sistemleri (ör. Redis)

gibi birden fazla kaynaktan sağlanmaktadır.

REST yaklaşımında söz konusu birleşik veri genellikle istemcinin birden fazla servise ayrı ayrı istek göndermesi veya arka uçta özel “toplayıcı” (aggregator) uç noktalar tanımlanması yoluyla sağlanır.

GraphQL’de ise bu tür birleştirme işlemleri resolver katmanında gerçekleştirilir. İstemci, tek bir sorgu ile çeşitli kaynaklardan toplanan verilere erişebilir. Özellikle mikroservis mimarilerinde, farklı servisleri tek ve tutarlı bir API yüzeyi altında toplamak için GraphQL etkili bir araçtır.

API Yüzeyinin Sık Evrildiği, Geriye Dönük Uyumun Önemli Olduğu Durumlar

Ürün geliştirme hızının yüksek olduğu ortamlarda alan adlarının değişmesi, yeni alanların eklenmesi veya bazı alanların devre dışı bırakılması sık karşılaşılan bir durumdur. REST tabanlı versiyonlama (/v1, /v2 vb.) bu değişiklikleri yönetmenin bir yoludur; ancak zamanla sürüm yönetimi karmaşık hâle gelebilir.

GraphQL’de alanlar bazında *deprecation* mekanizması kullanılarak, eski istemcilerin mevcut seçim kümeleriyle çalışmaya devam etmesi sağlanırken, yeni istemciler yeni alanları kullanabilir. Bu yaklaşım, API’nin hızlı evrilmesine karşın geriye dönük uyumluluğun korunmasını kolaylaştırır.

GraphQL Kullanımının Gerekli Olmayabileceği Durumlar

Basit ve Sınırlı CRUD Senaryoları

Sadece birkaç uç noktadan oluşan, ekranların veri ihtiyaçlarının açık ve sınırlı olduğu basit yönetim panelleri veya iç araçlar için GraphQL çoğu zaman zorunlu değildir. Bu tip uygulamalarda, iyi tasarlanmış REST uç noktaları hem geliştirme maliyetini hem de kavramsal karmaşıklığı düşük tutar.

Tek Tüketicili Servisler Arası İletişim

Sadece arka uç servislerinin birbiri ile haberleştiği, tek tüketicili senaryolarda çoğu zaman:

- REST,
- gRPC

gibi çözümler daha hafif ve yeterli olmaktadır. GraphQL'in esnek sorgulama gücü, birden fazla bağımsız istemci bulunmadığı durumlarda her zaman gerekli değildir.

Ekip Kapasitesinin Sınırlı Olduğu Durumlar

GraphQL ekosistemine geçiş, yalnızca sorgu dilini öğrenmekten ibaret değildir; şema tasarımı, code generation, önbellekleme stratejileri, hata yönetimi, N+1 sorgu probleminin çözümü gibi konularda da yatırım gerektirir.

Eğer ekibin mevcut yükü yüksekse ve bu alanlarda gerekli bilgi birikimi veya zaman yoksa, GraphQL'in yüzeysel ya da hatalı kullanımı, iyi tasarlanmış REST API'lerine kıyasla daha fazla karmaşa yaratabilir. Bu nedenle bazı durumlarda REST yaklaşımını iyi uygulamak, GraphQL'e geçmekten daha rasyonel bir tercih olabilir.

Karar Verirken Kullanılabilecek Kontrol Listesi

Aşağıdaki sorular, bir projede GraphQL kullanımının uygunluğunu değerlendirmek için pratik bir çerçeve sunmaktadır. Bu sorulardan üç veya daha fazlasına “evet” yanıtı veriliyorsa, GraphQL ciddi biçimde değerlendirilmeye değer bir aday olarak görülebilir:

1. Aynı arka uç sistemini kullanacak birden fazla istemci türü (web, mobil vb.) mevcut mu veya yakın gelecekte planlanıyor mu?
2. Aynı alan modelinden farklı ekranlar tarafından talep edilen, birbirinden farklı alan kombinasyonları bulunuyor mu?
3. Özellikle mobil ve yüksek gecikmeli ağ koşullarında, sunucuya yapılan istek sayısını azaltma ihtiyacı kritik önem taşıyor mu?
4. Veriler birden fazla kaynaktan (veritabanı, harici API, önbellek vb.) toplanmak zorunda mı?
5. API yüzeyi sık değişiyor mu ve buna rağmen mevcut istemcilerle geriye dönük uyumluluk korunmak isteniyor mu?
6. Ekip, GraphQL ekosistemine (şema tasarımı, araçlar, performans optimizasyonu vb.) yatırım yapabilecek zamana ve motivasyona sahip mi?

Adım Adım Modelleme ile Kapsamlı Senaryo Tasarımı Arasındaki İlişki

Bu bölümde, bir iş alanını (domain) veya kullanıcı deneyimini tasarlarken *tüm senaryoları önceden düşünme* ihtiyacı ile *GraphQL şemasını ve sistem modelini adım adım, senaryo bazlı inşa etme* yaklaşımının birbiriyle nasıl uyumlu olduğu ele alınmaktadır. İlk bakışta bu iki yaklaşım çelişkili görünebilse de, gerçekte farklı aşamalara ait iki tamamlayıcı pratikten ibarettir.

Keşif ve Tasarım Aşaması: Senaryoların Geniş Yazılması

Sistem tasarımı veya deneyim tasarımı yapılırken, öncelikli hedef, iş alanını ve kullanıcı yolculuklarını mümkün olduğunca kapsayıcı biçimde anlamaktır. Bu çerçevede:

- Kullanıcı yolculukları çıkarılır,
- Farklı kullanım senaryoları, akışlar ve uç durumlar (edge case) tanımlanır,
- Hatalı veya olağandışı durumlar düşünülerek sistemin nasıl davranması gerektiği tartışılır.

Bu aşama, esasen *düşünme* ve *keşif* aşamasıdır. Amaç, iş alanının sınırlarını, kullanıcı ihtiyaçlarını ve olasılık alanını geniş biçimde görmek ve belgelendirmektir. Bu nedenle burada “ne kadar çok senaryo varsa o kadar iyi” bakış açısı çoğu zaman geçerlidir.

Modelleme ve Uygulama Aşaması: İnce Dikey Dilimlerle İlerleme

GraphQL şeması, domain modeli veya teknik mimariyi inşa etme aşamasında önceki bölümde ortaya çıkarılan tüm senaryoların bir anda koda dökülmesi gerekmemektedir; hatta çoğu durumda bu tercih edilmemelidir. Bu noktada temel ilke şudur:

Tasarım aşamasında geniş düşünmek, modelleme ve uygulama aşamasında ise *ince, dikey dilimler* hâlinde ilerlemek daha sağlıklıdır.

Pratikte bu yaklaşım şu adımlarla somutlaşır:

1. Öncelikle, iş açısından kritik veya öncelikli olan *tek bir senaryo* veya *tek bir uçtan uca akış* seçilir (örneğin: “kullanıcı giriş yapar ve ana ekranı görür”).
2. Sadece bu senaryoyu baştan sona destekleyecek kadar GraphQL tipi, alanı ve ilişki tanımlanır.
3. Bu ilk model gerçek kullanım ve ekip geri bildirimi ile test edilir; isimlendirmelerin, ilişkilerin ve soyutlamaların uygunluğu gözlemlenir.
4. Ardından, bir sonraki senaryo için şema ve model kademeli olarak genişletilir.

Bu sayede, şemaya eklenen her öge belirli ve doğrulanmış bir ihtiyaca dayandığı için, modelin gereksiz karmaşıklık içermesi veya erken aşamada yanlış soyutlamalara kilitlenilmesi riski azaltılmış olur.

Tüm Senaryoları Baştan Koda Dökmemenin Gerekçeleri

Tasarım aşamasında yazılan senaryoların önemli bir kısmı, özünde *hipotez* niteliğindedir. Uygulama hayata geçtikçe:

- Bazı senaryoların hiç kullanılmadığı,
- Bazı senaryolarda kullanıcı davranışının beklenenden farklı olduğu,
- Bazı ilişkilerin veya kavramsal ayrımların yeniden düşünülmesi gerektiği

ortaya çıkabilmektedir. Bu nedenle:

1. Tüm senaryoların baştan GraphQL şemasına veya domain modeline kazınması, yanlış soyutlamaların API yüzeyine sızmasına ve sonradan değiştirmenin güçleşmesine yol açabilir.
2. İleride “belki lazım olur” düşüncesiyle modele eklenen her ek tip, alan veya ilişki, *spekülatif karmaşıklık* yaratır. Bu tür öğeler, gelecekte bakım maliyetini yükselten ve zihinsel yükü artıran unsurlar hâline gelebilir.

Bu sebeplerle, senaryoların planlama ve keşif aşamasında geniş tutulması, ancak kod seviyesinde sadece doğrulanmış veya öncelikli akışlar için modelleme yapılması daha dengeli bir yaklaşımdır.

Pratik Yaklaşım: Geniş Düşün, Dikey İnşa Et

Özetle, sağlıklı bir süreç aşağıdaki gibi yapılandırılabilir:

1. **Keşif ve tasarım:** İş alanına ilişkin tüm önemli senaryolar, kullanıcı yolculukları ve uç durumlar mümkün olduğunca kapsamlı şekilde yazılır.
2. **Önceliklendirme:** Bu senaryolar arasından iş değeri, teknik bağımlılıklar ve riskler dikkate alınarak ilk ele alınacak bir veya birkaç senaryo seçilir.
3. **Dikey dilim uygulama:** Seçilen senaryoyu uçtan uca (kullanıcı arayüzü, GraphQL şeması, domain modelleri ve kalıcı katman) destekleyecek kadar model ve implementasyon yapılır.
4. **Geri bildirim ve iyileştirme:** Gerçek kullanım, ekip değerlendirmesi ve ölçümlerle elde edilen geri bildirimler doğrultusunda, gerekirse isimlendirmeler, ilişkiler ve soyutlamalar erken safhada revize edilir.

5. **Kademeli genişletme:** Yeni senaryolar, aynı yaklaşım izlenerek sırayla ele alınır ve model kontrollü biçimde genişletilir.

Bu yaklaşım, hem tasarım aşamasında geniş perspektifi korumayı, hem de uygulama aşamasında kontrollü, doğrulanabilir adımlarla ilerlemeyi mümkün kılar.

Deneyim Tasarımına Yansıması

Benzer ilke, kullanıcı deneyimi tasarımına da doğrudan uygulanabilir. Yüksek seviyede tüm kullanıcı yolculuklarının ve ekranların haritalanması elbette değerlidir; ancak bileşenlerin, durum makinelerinin ve API entegrasyonlarının tasarımında, öncelikli akışlar üzerinden iteratif hareket etmek genellikle daha sağlıklı sonuç verir.

Erken aşamada çok soyut ve genel bileşenler tasarlamak, halen tam anlaşılmamış ihtiyaçlar için ağır soyutlamalar üretmeye ve sonrasında bu soyutlamalara bağımlı hâle gelmeye neden olabilir. Buna karşılık, önce kritik akışlar için net ve somut çözümler geliştirmek, daha sonra bu çözümlerden öğrenilenler ışığında yeniden kullanılabilir soyutlamalar çıkarmak daha güvenilir bir yöntemdir.

Sonuç

Sonuç olarak, “adım adım modelleme” ilkesi, tüm senaryoları düşünmeme çağrısı değil; tersine, kapsamlı senaryo tasarımını *uygulama seviyesinde kontrollü ve doğrulanabilir* adımlara bölme önerisidir. Tasarım ve keşif aşamasında geniş bir senaryo seti oluşturmak önemini korurken, GraphQL şeması, domain modelleri ve teknik mimari, iş açısından öncelikli ve doğrulanmış akışlar üzerinden dikey dilimler hâlinde inşa edilmelidir. Böylece hem yanlış tasarımlara erken aşamada büyük yatırım yapılması önlenir, hem de sistem gerçek kullanım verileriyle birlikte organik olarak olgunlaşabilir.

GraphQL'in HTTP Üzerinden Sunulması

Bu bölümde, GraphQL istek ve yanıtlarının HTTP protokolü üzerinden nasıl gönderilmesi ve alınması gerektiğine ilişkin temel ilkeler özetlenmektedir. GraphQL spesifikasyonu belirli bir taşıma protokolü dayatmasa da, pratikte yaygınlık ve altyapı desteği nedeniyle en sık kullanılan protokol HTTP'dir. Buradaki öneriler, stateless nitelikteki *query* ve *mutation* işlemleri için geçerlidir.

Genel Çerçeve ve Spesifikasyon

GraphQL dili, isteklerin nasıl taşınacağını değil, yalnızca sorgu yapısını ve yürütme semantiğini tanımlar. Buna paralel olarak gelişmekte olan *GraphQL-over-HTTP* taslak spesifikasyonu, HTTP üzerinden GraphQL API'lerinin nasıl sunulacağına dair ayrıntılı kurallar önermektedir. Bu taslak zorunlu olmamakla birlikte, birlikte çalışabilirlik (interoperability) amacıyla pek çok sunucu ve istemci kütüphanesi bu standartlara uyum sağlamaya yönelmektedir.

API Endpoint Tasarımı

REST yaklaşımı, URL'ler ile tanımlanan *resource* kavramına dayanırken, GraphQL'in temel modeli bir varlık grafidir. Bu nedenle GraphQL'de varlıklar URL ile değil, şema ve sorgu üzerinden tanımlanır. Tipik olarak:

- Bir GraphQL servisi, tek bir HTTP endpoint'i üzerinden sunulur.
- Bu endpoint çoğunlukla `/graphql` ile biter ve ilgili servise ait tüm GraphQL istekleri bu uç noktaya yönlendirilir.

Kimlik Doğrulama ve Yetkilendirme

Modern web çatıları, isteklerin bir dizi *middleware* veya filtre katmanından geçirilerek işlendiği bir *pipeline* modeli kullanır. Bu bağlamda:

- GraphQL endpoint'i, kimlik doğrulama (authentication) katmanlarından *sonra* konumlandırılmalıdır. Böylece diğer HTTP handler'larında olduğu gibi oturum ve kullanıcı bilgilerine erişim sağlanır.
- Yetkilendirme (authorization) kararları ise, GraphQL yürütmesi sırasında, özellikle resolver'lar içerisindeki iş mantığı katmanında verilmelidir. Bu sayede alan bazında yetkilendirme yapılabilir ve kısmi veri ve hata içeren yanıtlar üretmek mümkün olur.

İstek Formatı

Header'lar

GraphQL istemci ve sunucularının JSON serileştirmesini desteklemesi beklenir. Yanıtta hangi medya tiplerinin desteklendiğinin belirtilmesi için istemcinin `Accept` header'ını kullanması gerekir. Önerilen medya tipi:

- `application/graphql-response+json`

Geriye dönük uyumluluk amacıyla, özellikle 2025 öncesi sunucular için `application/json` tipi de eklenebilir:

- `Accept: application/graphql-response+json, application/json`

Encoding belirtilmediği durumda varsayılan olarak UTF-8 kabul edilir. `Accept` veya `Content-Type` header'larının eksik olması hâlinde sunucu 4xx statü kodu ile hata dönebilir.

HTTP Metodları

Bir GraphQL HTTP sunucusu:

- **POST** metodunu *query* ve *mutation* işlemleri için mutlaka desteklemelidir.
- **GET** metodunu ise, desteklemeyi tercih ederse, yalnızca *query* işlemleri için kabul etmelidir.

POST İstekleri

Standart bir GraphQL POST isteği:

- `Content-Type: application/json` header'ına sahip olmalı,
- Aşağıdaki yapıda JSON gövde içermelidir:

```
[style=code] {
  "query": "...",
  "operationName": "...",
  "variables": {
    "myVariable": "someValue"
  },
  "extensions": {
    "myExtension": "someValue"
  }
}
```

Burada:

- `query` alanı zorunludur ve tek bir GraphQL dokümanının kaynak metnini içerir (içinde `query`, `mutation` ve `fragment` tanımları bulunabilir).
- `operationName`, `variables` ve `extensions` alanları isteğe bağlıdır; yalnızca dokümanda birden fazla operasyon varsa `operationName` zorunlu hâle gelir.

GET İstekleri

GET yöntemi ile bir GraphQL sorgusu gönderildiğinde:

- GraphQL dokümanı, URL query string'inde `query` parametresi içinde iletilir, örneğin:

[style=code] GET /graphql?query=mename

Ek olarak:

- Değişkenler, JSON-encode edilmiş string olarak `variables` parametresinde,
- Birden fazla isimlendirilmiş operasyon varsa çalıştırılacak olan, `operationName` parametresiyle belirtilebilir.

HTTP Metodu Seçimi ve Önbellekleme

İstemcinin kullanabileceği HTTP metodları, sunucunun desteklediği metodlarla sınırlıdır. Ayrıca:

- Mutation işlemleri *her zaman* POST ile gönderilmelidir.
- Query işlemleri için GET desteği mevcutsa, HTTP veya CDN önbellekleme olanaklarından faydalanmak amacıyla GET tercih edilebilir.

Bununla birlikte, karmaşık GraphQL dokümanlarının metinleri uzun olabileceğinden, URL uzunluk sınırlarının (tarayıcı, CDN vb.) aşılması mümkündür. Bu durumlarda:

- Sunucunun, sorgu metinlerini bir kimlik (ID) ile eşleştiren *persisted documents*, *automatic persisted queries* veya *trusted documents* gibi teknikler kullanması,
- İstemcinin tam sorgu metni yerine kısa bir doküman kimliği göndermesi

pratik bir çözüm sunar.

Yanıt Formatı

Header

Sunucunun yanıtında kullanılacak medya tipi:

- Tercihen `application/graphql-response+json`,
- Geriye dönük uyumluluk için `application/json`

olabilir. `Content-Type` header'ında hem medya tipi hem de encoding (belirtilmemişse UTF-8) ifade edilmelidir.

Body

HTTP metodundan bağımsız olarak, GraphQL yanıt gövdesi JSON formatında döndürülmelidir. Genel yapı şöyledir:

```
"data": ... , "errors": [ ... ], "extensions": ...
```

Burada:

- `data`, başarılı yürütülen sorgunun sonuç verilerini içerir.
- `errors`, doğrulama veya yürütme sırasında ortaya çıkan hata nesnelerinin listesini içerir.
- `extensions`, implementasyona bağlı olarak ek meta bilgilerin taşınabildiği isteğe bağlı bir alandır.

Her zaman:

- Hiç hata yoksa `errors` alanı *bulunmamalıdır*.
- Yürütme başlamadan hata oluşmuşsa `errors` alanı mevcut, `data` alanı ise *bulunmamalıdır*.

HTTP Status Kodları

Statü kodu seçimi, hem yanıtın içeriğine hem de medya tipine göre şu ilkelere dayanır:

- Yanıtta `data` anahtarı mevcut ve değeri `null` değilse, hata nesneleri bulunsa dahi HTTP statü kodu 2xx olmalıdır. HTTP, “kısmi başarı” için ayrı bir kod tanımlamadığından bu yaklaşım benimsenir.
- Doğrulama hataları gibi, GraphQL operasyonunun yürütülmesini engelleyen durumlarda, sunucu genellikle 400 statü kodu döner.
- `application/json` ile yanıt veren eski sunucularda, geçerli GraphQL isteklerinin yürütülememesi hâlinde 4xx veya 5xx kodları kullanılması mümkün olmakla birlikte, bu durum belirsizlik yaratabileceği için genellikle `2xx + errors` alanı tercih edilir.
- `application/graphql-response+json` medya tipi kullanıldığında, geçerli bir GraphQL isteği yürütülememişse sunucunun 4xx veya 5xx statü kodu ile yanıt vermesi beklenir.

Özet

HTTP üzerinden GraphQL servisi sunarken önerilen başlıca ilkeler şöyle özetlenebilir:

- Kimlik doğrulama, GraphQL isteği doğrulanmadan önce HTTP pipeline'ında yapılmalı; yetkilendirme ise GraphQL yürütmesi sırasında iş mantığı katmanında gerçekleştirilmelidir.
- GraphQL API'leri genellikle tek bir endpoint üzerinden sunulur ve bu endpoint çoğunlukla `/graphql` ile biter.
- İstemci ve sunucu JSON formatını desteklemeli; istemci, yanıt medya tipini `Accept` header'ında özellikle `application/graphql-response+json` ile belirtmelidir.
- GraphQL istekleri varsayılan olarak POST metoduyla, `Content-Type: application/json` header'ı eşliğinde gönderilmelidir; query operasyonları için isteğe bağlı olarak GET de desteklenebilir.
- Yürütülen GraphQL operasyonunun tam veya kısmi sonucu, JSON yanıt gövdesinde `data` anahtarı altında; doğrulama ve yürütme sırasında oluşan hatalar ise `errors` anahtarı altında sunulmalıdır.
- Geçerli bir GraphQL isteğine karşılık dönen yanıtta `data` alanı `null` değilse, HTTP statü kodu 2xx olmalı; yürütülmeyen veya başarısız olan isteklerde ise kullanılacak statü kodu, yanıtın medya tipine ve sunucu implementasyonuna göre 4xx/5xx ile farklılık gösterebilir.

GraphQL ile Dosya Yüklemenin Sınırları ve Önerilen Yaklaşım

Bu bölümde, dosya yükleme işlemlerinin neden GraphQL üzerinden gerçekleştirilmesinin uygun olmadığı ve bunun yerine hangi yaklaşımın tercih edilmesi gerektiği özetlenmektedir.

GraphQL ile Dosya Yüklemenin Zorluğu

GraphQL temelde:

- Görece küçük boyutlu,
- JSON gibi metin tabanlı serileştirme formatlarıyla taşınan

istek ve yanıtlar için tasarlanmıştır. Buna karşılık, dosya yükleme işlemleri genellikle resim, PDF veya video gibi *binary* verilerle çalışır.

JSON, ikili veriyi doğrudan taşıyamadığı için ek kodlama (örneğin base64) gerektirir. Bu yaklaşım:

- Verimsizdir (mesaj boyutunu artırır),
- Büyük dosyalar için uygun değildir,
- Akış tabanlı (streamed) işleme imkânlarını kısıtlar.

HTTP tarafında ikili veri için yaygın çözüm `multipart/form-data` kullanımıdır; ancak bu formatın GraphQL ile birleştirilmesi hem uygulama karmaşıklığını artırmakta hem de güvenlik ve dayanıklılık açısından ek riskler getirmektedir.

Başlıca Riskler

Bellek Tükenmesi ve Değişken Tekrarı

GraphQL sorgularında aynı değişkenin birden fazla yerde kullanılmasına izin verilir. Eğer bu değişken bir dosya yüklemesini temsil ediyorsa, alttaki akışın (stream):

- Birden fazla kez okunmaya çalışılması,
- Erken tüketilmesi veya yanlış yönetilmesi

sonucunda yanlış davranışlar veya bellek tükenmesi (memory exhaustion) oluşabilir.

Başarısız İsteklerde Akışların Açık Kalması

GraphQL yürütmesi, *doğrulama* ve *çalıştırma* olmak üzere iki fazda gerçekleşir. Eğer doğrulama hataları veya yetkilendirme sorunları nedeniyle yürütme başlamadan istek başarısız olursa, gelen dosya akışları hiç tüketilmeden açık kalabilir. Sunucu bu akışları buffer'lıyor veya bellekte tutuyorsa, bu durum kaynak sızıntılarına ve bellek kullanımının artmasına yol açar.

CSRF Riskleri

`multipart/form-data`, CORS spesifikasyonu açısından “basit” (simple) istek kapsamındadır ve preflight kontrolü tetiklemeyebilir. Ek bir CSRF koruması olmadan, GraphQL sunucusu kötü niyetli kökenlerden gelen dosya yükleme isteklerini farkında olmadan kabul edebilir.

Aşırı Büyük veya Gereksiz Yükler

Saldırganlar çok büyük boyutlu dosyalar gönderebilir veya kullanılmayan değişken adları altında gereksiz dosyalar ekleyebilir. Bu tür payload’lar, isteği kabul eden ve tamponlayan sunucuların kaynaklarının hızla tükenmesine neden olabilir.

Güvenilmeyen Dosya Metaverisi

Dosya adı, MIME türü ve içerik gibi öznitelikler hiçbir zaman doğrudan güvenilmemelidir. Aksi hâlde:

- Yol geçişi (path traversal) ve enjeksiyon saldırıları,
- Yanlış veya kötü niyetli MIME tipleri,
- Zip bomb, kötü amaçlı PDF gibi format bazlı saldırılar

gibi riskler ortaya çıkabilir.

Önerilen Yaklaşım: İmzalı URL (Signed URL) Kullanımı

En güvenli ve ölçeklenebilir çözüm, dosya yükleme işlemlerini GraphQL'in dışına taşımaktır. Önerilen model şu adımlardan oluşur:

1. İstemci, bir GraphQL mutasyonu ile depolama sağlayıcısından (örneğin Amazon S3) dosya yüklemek için kullanılacak *imzalı bir URL* talep eder.
2. İstemci, dosyayı bu imzalı URL'yi kullanarak doğrudan depolama servisine yükler (GraphQL bu adımda devrede değildir).
3. Yükleme tamamlandıktan sonra:
 - İkinci bir GraphQL mutasyonu ile ilgili dosyanın uygulama verisiyle ilişkilendirilmesi sağlanır, veya
 - Depolama tarafında otomatik tetiklenen bir süreç (örneğin Lambda fonksiyonu) bu ilişkilendirmeyi yapar.

Bu yaklaşım:

- GraphQL sunucusunu doğrudan ikili veri trafiğinden izole eder,
- Sorumlulukları net biçimde ayırır (GraphQL: iş mantığı ve metadata; depolama servisi: büyük dosya trafiği),
- Modern web mimarilerinde önerilen en iyi uygulamalarla uyumlu bir çözüm sunar.

Geçici yükleme alanının kısa ömürlü tutulması, saldırganların yalnızca imzalı URL alıp depolama alanını doldurmasını önlemeye yardımcı olur. Dosya işlendikten sonra uygun kalıcı depoya taşınmalı, kullanılmayan yüklemeler ise temizlenmelidir.

GraphQL’de Yetkilendirme (Authorization) ve İş Mantığı Katmanı

Bu bölümde, GraphQL tabanlı bir sistemde yetkilendirme (authorization) sorumluluğunun neden doğrudan GraphQL katmanına değil, iş mantığı (business logic) katmanına devredilmesi gerektiği ele alınmaktadır. Kimlik doğrulama (authentication) ile yetkilendirme (authorization) arasındaki sorumluluk paylaşımı ve bu yaklaşımın pratik sonuçları tartışılmaktadır.

Kimlik Doğrulama ve Yetkilendirme Ayrımı

Bir GraphQL isteği sunucuya ulaştığında işlem hattı genel olarak şu şekilde yapılandırılmalıdır:

- **Kimlik doğrulama (authentication):** İstek, GraphQL yürütmesi başlamadan önce bir *middleware* zincirinden geçer. Bu aşamada kullanıcının kimliği doğrulanır ve ilgili kullanıcı bilgisi, GraphQL isteğine ait `context` nesnesine eklenir (örneğin `context.user`).
- **Yetkilendirme (authorization):** Kullanıcı doğrulandıktan sonra, kullanıcının belirli veri alanlarına veya işlemlere erişim hakkının olup olmadığına karar verilmesi gerekir. Bu kararlar, GraphQL katmanına dağılmak yerine iş mantığı katmanında verilmelidir.

Bu ayırım sayesinde, doğrulama ve yetkilendirme sorumlulukları farklı aşamalara bölünerek hem mimari netlik sağlar hem de kuralların tek bir yerde yönetilmesi kolaylaşır.

- Aynı yetkilendirme kuralına ihtiyaç duyan tüm giriş noktalarında (farklı API’ler, farklı servisler vb.) benzer kodların *tekrar tekrar* yazılması gerekir.
- Eğer bu kodlar zaman içinde senkronize tutulamazsa, aynı iş kuralı farklı yerlerde farklı biçimlerde uygulanır ve kullanıcılar, hangi giriş noktasını kullandıklarına bağlı olarak farklı sonuçlar görebilir.

Dolayısıyla, resolver içinde doğrudan yetkilendirme mantığı tutmak, tutarsızlıklara ve güvenlik açıklarına zemin hazırlayabilir.

Doğru Yaklaşım: Yetkilendirme Mantığının İş Katmanına Delegasyonu

Önerilen mimari, yetkilendirme kurallarının iş mantığı katmanında tek bir yerde tanımlanmasıdır. GraphQL resolver'ları ise bu kuralları çağıran, nispeten ince geçiş noktaları olarak kalmalıdır.

İş mantığı katmanına mümkünse yalnızca bir token veya anahtar değil, *tam olarak hydrate edilmiş* bir kullanıcı nesnesi aktarılması önerilir. Böylece:

- Kimlik doğrulama (token çözümleme) aşaması middleware katmanında,
- Yetkilendirme (bu kullanıcının neleri yapabileceğine karar verme) aşaması ise iş mantığı katmanında gerçekleşir.

Tüm yetkilendirme mantığının iş mantığı katmanında tutulması genel olarak en sağlıklı yaklaşım olsa da, şema seviyesinde bazı alanların hangi yetki kurallarına tabi olduğunu ifade etmek için *type system directive* kullanımı düşünülebilir.

Özet

GraphQL ile yetkilendirme yapılırken aşağıdaki ilkeler benimsenmelidir:

- Yetkilendirme mantığı, GraphQL resolver'larının içine dağılmak yerine, iş mantığı katmanında tek bir kaynaktan toplanmalıdır.
- Kimlik doğrulama, GraphQL yürütmesi başlamadan önce middleware aşamasında yapılmalı; GraphQL yürütmesi sırasında ise hangi alanlara erişileceğine ilişkin kararlar, iş mantığı katmanındaki kurallar temelinde verilmelidir.
- Şema üzerinde directive kullanımıyla, belirli alan ve tiplerin hangi genel yetki kurallarına tabi olduğu deklaratif olarak belirtilebilir; ancak bu, yetkilendirme mantığının iş mantığı katmanında tutulması ilkesini değiştirmez.

GraphQL’de Sayfalandırma (Pagination)

Bu bölümde, GraphQL şemasında büyük listelerin tutarlı ve esnek bir şekilde sayfalandırılması için önerilen *cursor tabanlı connection* modelinin temel kavramları özetlenmektedir.

Liste Alanları ve Temel Kullanım

GraphQL’de nesneler arası ilişkiler çoğunlukla liste alanları ile ifade edilir. Örneğin, bir karakterin arkadaşlarını almak için:

```
hero {  
  name  
  friends {  
    name  
  }  
}
```

şeklinde bir sorgu kullanılabilir. Ancak, bu alan büyük sayıda eleman döndürebileceği için *sayfalandırma* ihtiyacı doğar. İlk adım olarak, kaç adet eleman istenildiğini belirtmek mümkündür:

```
hero {  
  name  
  friends(first: 2) {  
    name  
  }  
}
```

Offset Tabanlı ve Cursor Tabanlı Sayfalandırma

Sayfalandırma için farklı yaklaşımlar mümkündür:

- Offset tabanlı: `friends(first: 2, offset: 2)`
- Cursor tabanlı: `friends(first: 2, after: $friendCursor)`

Offset tabanlı yaklaşım, özellikle büyük veri kümelerinde performans ve güvenlik açısından zayıflıklara sahiptir ve yeni kayıtlar eklendiğinde sayfa sınırları kayarak belirsizlik yaratabilir. Buna karşılık, **cursor tabanlı sayfalandırma**:

- Daha stabil ve esnektir,
- Cursor'lar *opak* tutulduğunda (örneğin base64 ile kodlanmış) backend'in sayfalandırma stratejisi zaman içinde değiştirilebilir,
- Offset tabanlı modeller de, uygun cursor tanımı ile bu yapı üzerinden uygulanabilir.

Sosyal Medya Akışlarında Sayfalandırma ve Yenileme Davranışı

Bu bölümde, sosyal medya uygulamalarında veri tabanının fiilen “dondurulmadığı”, ancak istemcinin (mobil veya web uygulamasının) sayfalandırma (*pagination*) ve yenileme (*pull-to-refresh*) davranışlarıyla sanki sabit bir görünüm sunuyormuş gibi hareket ettiği özetlenmektedir. Özellikle cursor tabanlı sayfalandırma, kullanıcının “nerede kaldığı” bilgisini korurken, sistemin arka planda akmaya devam etmesine imkân verir.

Aşağı Doğru Kaydırma: Eski Gönderilere Cursor Tabanlı Erişim

Kullanıcı, akışta aşağıya doğru kaydırıldığında:

- Uygulama, arka uca genellikle `first` ve `after` parametreleri ile *cursor tabanlı* istekler gönderir (örneğin GraphQL'de `posts(first: 20, after: "CURSOR_100")`).
- Her sayfa isteğinde, istemci bir önceki yanıtın `endCursor` değerini “nerede kaldığını” temsil eden referans olarak kullanır.
- Veri tabanına yeni kayıtlar eklense dahi, istemci “*şu kayıttan sonra gelenler*” mantığıyla ilerlediği için, aşağı doğru sayfalar tutarlı kalır; kullanıcı aynı gönderiyi tekrar görmez ve kayıtları atlamaz.

Bu sayede, veri tabanı akışkan ve güncel kalırken, kullanıcının aşağıya doğru ilerlediği “kişisel zaman çizgisi” cursor'lar sayesinde stabil bir şekilde korunur.

Yukarı Doğru Çekerek Yenileme: Pull-to-Refresh Davranışı

Kullanıcı akışın en üstündeyken “pull-to-refresh” (yukarı çekip bırakma) jestini kullandığında, uygulama genellikle aşağıdaki iki yaklaşımdan birini benimser:

Tam Yeniden Yükleme

- İstemci, akışın en güncel hâlini almak için baştan yeni bir istek gönderir (örneğin `posts(first: 20)`).
- Eski liste, gelen yeni sonuçla tamamen değiştirilir.
- Sıralama veya puanlama mantığı değişmişse, kullanıcı daha önce gördüğü bazı gönderilerin yer değiştirdiğini veya artık görünmediğini fark edebilir.

Bu davranış, kullanıcıya “liste sil baştan yenilenmiş” hissi verir; ancak veri tabanının dondurulması söz konusu değildir, yalnızca yeni bir sorgu çalıştırılmıştır.

Birleştirerek Güncelleme

- Uygulama, en üstteki gönderinin cursor veya kimliğini (*top cursor*) tutar.
- Yenileme jesti sonrasında, bu cursor’dan daha yeni olan gönderileri ister (örneğin “*bu noktadan daha yeni olanları getir*”).
- Gelen yeni gönderiler mevcut listenin en üstüne eklenir; alt taraftaki eski gönderiler aynı sırada kalmaya devam eder.

Bu yaklaşımda, ekran baştan boyanmak yerine, kullanıcı sadece “tepeye yeni satırlar eklendiğini” görür.

Gerçekte Ne “Dondurulmuş” Gibi Davranır?

Tek “dondurulan” şey veri tabanının kendisi değildir; dondurulan, istemcinin *nerede kaldığına dair referansıdır*. Kullanıcı aşağı kaydıkça:

- Uygulama, örneğin `CURSOR_100` gibi bir değeri, “100 numaralı gönderiye kadar gösterildi” anlamında referans olarak saklar.
- Sonraki isteklerde, `after: "CURSOR_100"` diyerek bu noktadan sonra gelenleri talep eder.

Veri tabanına yeni kayıtlar eklense bile, istemci “*bu cursor’dan sonraki dünya*” ile ilgilendiği için, kullanıcının aşağıya doğru sayfaladığı bölge tutarlı görünür. Bu sayede, sistem hem sürekli güncel kalır, hem de kullanıcılar sayfalandırma sırasında tekrarlayan veya atlanan kayıtlarla karşılaşmadan akıcı bir deneyim yaşayabilir.

Node, Edge ve Connection Kavramlarının Basit Açıklaması

Bu bölümde, GraphQL'deki *node*, *edge* ve *connection* kavramları, basit bir Netflix / dizi listesi benzetimi üzerinden açıklanmaktadır.

Node: Filmin Kendisi

Önce en temel nesneyi düşünelim: *film*.

Örneğin elimizde şu liste olsun:

Filmler:

- Inception
- Shrek
- Interstellar

Burada her bir film birer **node** olarak düşünülebilir. GraphQL tipinde bu, kabaca şöyle temsil edilir:

```
type Movie {  
  id: ID!  
  title: String!  
}
```

Yani *node*:

- Film veya kullanıcı gibi asıl varlığı (entity) temsil eden nesnedir,
- Örneğin Movie, User veya Post gibi tiplerdir.

Edge: Sen ve Film Arasındaki Satır (İlişki)

Netflix'teki “*Continue Watching / İzlemeye Devam Et*” listesini düşünelim. Bu listede sadece filmin adı değil, aynı zamanda:

- Bu filmi yüzde kaç izlediğin (progress),
- Son izleme zamanın,
- Bu liste içinde sırası

gibi bilgiler de yer alır.

Bu durumda:

- Filmin kendisi (id, title)= **node**,
- *Sen + o film* arasındaki ilişki satırı (izleme ilerlemesi vb.)= **edge**.

Bunu basit bir JSON benzetimiyle gösterebiliriz:

```
[
  {
    "movie": { "id": 1, "title": "Inception" },    // node
    "progress": 0.8,                               // edge bilgisi
    "cursor": "CURSOR_1"                          // edge için sayfalama anahtarı
  },
  {
    "movie": { "id": 2, "title": "Shrek" },        // node
    "progress": 0.2,                               // edge bilgisi
    "cursor": "CURSOR_2"                          // edge için sayfalama anahtarı
  }
]
```

Burada:

- movie alanı, asıl nesneyi (node) temsil eder.
- progress ve cursor, bu node'un *bu liste içindeki* ilişkisine özgü edge bilgileridir.

Connection: Listenin Tamamı ve Meta Bilgi

Şimdi Netflix ana ekranındaki bir satırı düşünelim: “*Continue Watching*” satırı.

Bu satır:

- İçinde bir dizi kart (film) barındırır,
- Sağda/altında toplam kaç film olduğu,
- Devam edip edemeyeceğin (daha fazla var mı) gibi bilgiler

hakkında da bilgi sunabilir.

İşte bu satırın tamamı, GraphQL terminolojisinde bir **connection** olarak düşünülebilir.

Bunu JSON benzetimiyle şu şekilde ifade edebiliriz:

```
{
  "totalCount": 12,           // listede toplam 12 film var
  "edges": [
    {
      "movie": { "id": 1, "title": "Inception" },
      "progress": 0.8,
      "cursor": "CURSOR_1"
    },
    {
      "movie": { "id": 2, "title": "Shrek" },
      "progress": 0.2,
      "cursor": "CURSOR_2"
    }
  ],
  "pageInfo": {
    "endCursor": "CURSOR_2",
    "hasNextPage": true
  }
}
```

Bu yapıda:

- edges: Her eleman için *node* + ilişkiye özgü ek bilgiler (progress, cursor vb.).
- pageInfo: Sayfalandırma hakkında bilgi (endCursor, hasNextPage gibi).
- totalCount: Bu connection içindeki toplam film sayısı.

Dolayısıyla:

- **Node**= Filmin kendisi (örneğin `Movie` nesnesi),
- **Edge**= Bu film ile senin arandaki ilişki satırı (izleme ilerlemesi, `cursor` vb.),
- **Connection**= Edge'lerin tamamı + sayfalandırma ve istatistik bilgileri (`pageInfo`, `totalCount`).

GraphQL Şema Tasarımı: Versiyonlama ve Nullability

Bu bölümde, GraphQL şema tasarımında iki temel kavram özetlenmektedir: *versiyonlama yaklaşımı* ve *nullability* (alanların boş/“null” olabilme durumu).

Versiyonlama Yerine Şemanın Evrimleştirilmesi

Geleneksel API tasarımlarında (özellikle REST):

- Endpoint’in döndürdüğü veri üzerinde istemci tarafında sınırlı kontrol vardır.
- Küçük bir değişiklik dahi mevcut istemcileri bozabileceğinden *breaking change* olarak kabul edilir.
- Bu nedenle, değişiklikler genellikle /v1, /v2 gibi yeni API versiyonları açılarak yönetilir.

Bu yaklaşım, şu ikilemi doğurur:

- Sık sık, küçük versiyonlar yayımlamak: Esnek ama takip etmesi zor ve karmaşık bir API yüzeyi.
- Seyrek ve büyük versiyonlar yayımlamak: Daha az karmaşa ama esnekliği düşük, yavaş evrilen bir API.

GraphQL’de ise:

- İstemci yalnızca *açıkça talep ettiği* alanları alır.
- Sunucu, şemaya yeni tipler veya mevcut tiplere yeni alanlar ekleyebilir; eski istemciler bu alanları istemediği sürece davranışları değişmez.
- Bu sayede yeni kabiliyetler genellikle *kırıcı olmayan* (non-breaking) eklemelerle sağlanır.

Sonuç olarak yaygın pratik:

- Kırıcı değişiklik yapmaktan mümkün olduğunca kaçınmak,
- Şemayı sürekli, küçük eklemelerle *evrimleştirmek*,
- Ayı versiyon numaraları sunmak yerine, *versionless* (tek sürümlü) bir GraphQL API sunmaktır.

Nullability: Varsayılan Olarak Nullable Alanlar

Pek çok tip sisteminde, null desteği şu şekilde modellenir:

- Temel tip (`T`) ve bunun *nullable* versiyonu (`T?`) ayrıdır.
- Varsayılan tip genellikle “null içermez” kabul edilir; null desteği ayrıca belirtilir.

GraphQL’de yaklaşım tersine çevrilmiştir:

- **Her alan, varsayılan olarak nullable’dır.**
- Bunun nedeni, dağıtık bir sistemde alan bazında çok sayıda hata kaynağı bulunabilmesidir:
 - Veritabanı erişimi başarısız olabilir,
 - Arka plandaki bir servis cevap vermeyebilir,
 - Çalışma zamanı hatası (exception) oluşabilir,
 - Alan bazında farklı yetkilendirme kuralları devreye girebilir.

Bu varsayılan tasarım sayesinde:

- Bir alanın alınması sırasında hata oluştuğunda, tüm isteği başarısız saymak yerine sadece ilgili alan `null` döndürülebilir.
- İstek genel olarak başarılı kalırken, hata detayları `errors` alanında raporlanabilir.

GraphQL ayrıca *non-null* tipler de sunar:

- Örneğin `String!` veya `Int!`.
- Bu tipler istemciye şu garantiyi verir:

“Bu alan hiçbir zaman `null` olmayacak; eğer hata oluşursa, bunun bir üstindeki parent alan `null` olarak işaretlenecek.”

Şema tasarlarken şu soruyu sormak önemlidir:

- Bu alan için hata durumunda `null` dönmek, iş mantığı açısından makul mü?
 - Çoğu alan için cevap evet olacaktır ve varsayılan nullable davranış yeterlidir.
 - Bazı kritik alanlar için ise `null` kabul edilemez; bu durumda non-null (!) kullanılarak daha güçlü bir garanti verilir.

Özetle GraphQL, hem API yüzeyini *versiyonsuz* biçimde evrimleştirmeye uygun bir model sunar, hem de varsayılan nullable alanlar üzerinden isteklerin tamamını patlatmadan, alan bazında hataları tolere edebilen esnek bir hata modeli sağlar.

Connection Nesnesi ve pageInfo

Sadece sayfadaki elemanları değil, bağlantının kendisi hakkında da bilgi almak için *connection* tipi kullanılır. Bu tip tipik olarak:

- `edges`: Edge listesini,
- `totalCount`: Toplam eleman sayısını,
- `pageInfo`: Sayfalama durumu hakkında bilgiyi

içerir. Örnek bir sorgu:

```
friends(first: 2) {  
  totalCount  
  edges {  
    node { name }  
    cursor  
  }  
  pageInfo {  
    endCursor  
    hasNextPage  
  }  
}
```

PageInfo tipinde genellikle:

```
type PageInfo {  
  startCursor: ID  
  endCursor: ID  
  hasNextPage: Boolean!  
}
```

gibi alanlar bulunur. Böylece istemci:

- Bir sonraki sayfa için hangi cursor'ı kullanacağını bilir,
- Başka bir sayfa isteği yapmadan *devam var mı* sorusunun cevabını alır.

Bazı tasarımlarda, connection tipi içinde `friends` veya `nodes` gibi doğrudan düğüm listesini döndüren yardımcı alanlar da eklenerek, `edges` katmanı ile uğraşmak istemeyen istemcilere kolaylık sağlanır.

Caching ve ID

GraphQL’de caching yapabilmek için, her nesneye tutarlı bir id vermemiz yeterlidir. Bu id değeri, istemcinin o nesneyi cache’te tanıması için kullanılan anahtar (*cache key*) olur.

Yani özetle: “*Cache’lemek istiyorsam, her nesne için değişmeyen bir id üretirim ve bunu dışarı açarım.*”

Performance

GraphQL istekleri ilk bakışta zor cache’lenebilir görünse de, doğru tekniklerle REST kadar (hatta bazı durumlarda daha da) verimli hale getirilebilir. Aşağıda, GraphQL API performansını iyileştirmek için önerilen temel yaklaşımlar özetlenmektedir.

İstemci Tarafı Cache

GraphQL istemcileri (örn. Apollo, Relay) genellikle yerleşik cache mekanizmalarına sahiptir. Bu cache sayesinde:

- Aynı nesne birden fazla sorguda geldiğinde, id ve __typename gibi alanlara bakarak tek bir cache kaydı altında birleştirilebilir.
- Daha önce alınmış veriler tekrar sunucudan çekilmeden, doğrudan cache üzerinden okunabilir.
- UI, ağ gecikmesine daha az bağımlı ve daha tutarlı bir şekilde güncellenebilir.

Sorgular için GET Kullanımı

GraphQL sunucuları varsayılan olarak POST isteklerini destekler; ancak sadece *query* operasyonları için GET methodu da kullanılabilir.

- GET istekleri HTTP ve CDN katmanında cache’lenebilir olduğundan, performans açısından avantaj sağlar.
- Sorgu, URL içindeki *query* parametresiyle gönderilebilir; ancak URL uzunluk sınırları nedeniyle karmaşık sorgular için bu pratik olmayabilir.

Bu sorunu aşmak için *persisted queries* veya *hashed queries* kullanılabilir:

- İstemci, tam sorgu yerine sorgunun hash değerini gönderir.
- Sunucu, bu hash ile önceden kayıtlı sorguyu bulur, doğrular ve çalıştırır.
- Böylece:

- Daha kısa URL’ler elde edilir,
- HTTP/CDN cache kullanımı kolaylaşır,
- Ağ üzerinden gönderilen veri miktarı azalır.

N+1 Problemi

GraphQL, her field için ayrı resolver yazmaya imkân tanıyarak sunucu kodunu temiz ve modüler hale getirir. Ancak bu durum, ek bir önlem alınmadığında *N+1 problemine* yol açabilir.

- Örnek: Bir hero nesnesi, onun friends listesi ve her arkadaş için starships alanı sorgulandığında:

- 1 istek: kahramanı (hero) almak,
- 1 istek: arkadaş listesini almak,
- N istek: her arkadaş için starship listesini almak,

şeklinde toplam N+2 arka uç isteği oluşabilir.

Bu problem genellikle *batching* ve *caching* teknikleriyle çözülür:

- Facebook’un DataLoader kütüphanesi gibi araçlarla, aynı tipteki birden çok istek kısa bir zaman aralığında toplanır.
- Bu istekler tek bir WHERE id IN (...) sorgusu gibi toplu bir çağrıya dönüştürülerek veritabanına veya microservice’e gönderilir.
- Böylece N ayrı çağrı yerine 1 toplu çağrı yapılarak gecikme ve yük azaltılır.

Demand Control (İstek Karmaşıklığını Sınırlama)

Bazı GraphQL şemaları, istemcilerin istemeden ya da kötü niyetle aşırı karmaşık ve maliyetli sorgular gönderebilmesine izin verebilir. Bu tür durumlar, arka uç kaynakları üzerinde gereksiz yük oluşturur.

Bunu önlemek için:

- **Pagination:** Büyük listeler daima sayfalara bölünmeli, sonsuz uzunlukta listeler tek istekte döndürülmemelidir.
- **Derinlik sınırlaması:** Bir sorgunun ağaç derinliği (nesting seviyesi) belirli bir maksimum değerin üzerinde kabul edilmemelidir.
- **Genişlik / karmaşıklık analizi:** Bir sorgunun toplam alan sayısı veya “karmaşıklık puanı” hesaplanarak, belirli bir eşiğin üzerindeki istekler reddedilebilir.
- **Oran sınırlama (rate limiting):** Belirli bir kullanıcı veya API anahtarı için isteklere kota ve hız limiti getirilebilir.

JSON ve GZIP

GraphQL yanıt formatı spesifikasyon tarafından zorunlu tutulmasa da, uygulamada yanıtlar neredeyse her zaman JSON formatında döndürülür.

- JSON metin tabanlı olduğu için, GZIP, deflate ya da brotli gibi sıkıştırma algoritmalarıyla çok yüksek oranda sıkıştırılabilir.
- Üretim ortamında:
 - Sunucunun GZIP (veya benzeri) sıkıştırmayı etkinleştirmesi,
 - İstemcinin `Accept-Encoding: gzip` header'ını göndermesi önerilir.
- Bu sayede, ağ üzerinden taşınan payload boyutu ciddi ölçüde azalır ve yanıt süreleri iyileşir.

Ayrıca JSON:

- Geliştiriciler için çok tanıdık bir format olup,
- Kolay okunur ve debug edilir,
- GraphQL sorgu söz dizimi de kısmen JSON'dan ilham alır.

Performans İzleme ve Gözlemlenebilirlik

GraphQL API'nin zaman içindeki sağlığını korumak ve olası darboğazları tespit etmek için performans izlemesi (monitoring) büyük önem taşır.

- Belirli alanların sürekli yavaş yanıt verdiği,
- Bazı alanların sık sık hata ürettiği,
- Belirli sorguların olağan dışı kaynak tüketimine yol açtığı

gibi durumlar, metrikler, loglar ve izler (traces) toplanarak tespit edilebilir.

OpenTelemetry gibi gözlemlenebilirlik araçları:

- Farklı dillerde GraphQL sunucularını enstrümante etmeye,
- İstek süresi, hata oranı, alan bazlı gecikme gibi metrikler toplamaya,
- Bu verileri çeşitli izleme sistemlerine iletmeye olanak sağlar.

Security

GraphQL, diğer API türleri gibi kötü niyetli isteklerden ve aşırı yükten korunması gereken bir katmandır. Özellikle karmaşık ve ağır sorgularla yapılabilecek *denial of service* tarzı saldırılara karşı ek önlemler almak gerekir. Aşağıda GraphQL API'leri için temel güvenlik yaklaşımları özetlenmektedir.

Transport Katmanı Güvenliği

GraphQL spesifikasyonu belirli bir taşıma protokolü zorunlu kılmaz, ancak çoğu uygulama sorgu ve mutasyonlar için HTTP, subscription için WebSocket veya Server-Sent Events (SSE) kullanır. Seçilen protokol ne olursa olsun:

- HTTP kullanılıyorsa **HTTPS** ile trafiğin şifrelenmesi,
- İstekler için makul **timeout** değerleri belirlenmesi,
- HTTP cache kullanılıyorsa hassas verilerin *public* yerine *private* veya hiç cache'lenmemesi

gibi genel API güvenlik uygulamaları GraphQL için de birebir geçerlidir.

Demand Control: İstek Yükünü Kontrol Etmek

Amaç, tek bir GraphQL sorgusunun sunucuyu aşırı yüklemesini veya bilerek istismar edilmesini önlemektir. Bunun için birkaç tamamlayıcı teknik kullanılabilir.

Trusted Documents (Allowlist Yaklaşımı)

Eğer API yalnızca *first-party* istemciler tarafından kullanılıyorsa:

- Geliştirme ortamında sorguları geliştiriciler yazar.
- Build aşamasında bu sorgular sunucuya gönderilir ve bir *allowlist* (güvenilir sorgular listesi) içinde hash veya ID ile saklanır.
- Üretim ortamında istemci, tam sorgu metni yerine sadece bu *document ID* veya hash'i gönderir.
- Sunucu, sadece tanıdığı (allowlist'te olan) sorguları çalıştırır.

Böylece, istemcinin kafasına göre yeni GraphQL dokümanı göndermesi engellenmiş olur. Bu teknik, üçüncü parti istemcilere açık *public* API'ler için uygun değildir; ancak kapalı ekosistemler için güçlü bir savunma hattı sağlar.

Paginated Fields

Liste döndüren alanlar, tek istekte çok büyük veri setleri döndürebilir. Bu nedenle:

- Her zaman sayfalamaya (*pagination*) zorlanmalı,
- Örneğin `friends` yerine `friends(first: 10)` veya `friendsConnection(first: 20)` gibi limitli varyantlar kullanılmalıdır.

Bu sayede tek istekte milyonlarca kaydın dönmesi engellenir ve sunucu kaynakları korunur.

Depth Limiting (Derinlik Sınırı)

GraphQL'in gücü, ilişkisel yapıları derin bir şekilde ifade edebilmesidir; ancak bu, kötü yazılmış veya kötü niyetli sorgularla ağaç derinliğinin kontrolden çıkmasına sebep olabilir:

- Örneğin `friends { friends { friends { ... } } }` şeklinde recursive veya aşırı iç içe geçmiş selection set'ler CPU ve bellek kullanımını arttırabilir.

Bu nedenle:

- Her sorgu için maksimum bir *derinlik* (*max depth*) belirlenmesi,
- Bu sınırı aşan dokümanların, daha *validation* aşamasında hata ile geri çevrilmesi,
- Özellikle iç içe listelerin derinliği için daha düşük bir ek limit uygulanması

önerilir.

Breadth ve Batch Limiting (Genişlik ve Toplu İstek Sınırı)

Sorgu ağaç derinliği düşük olsa bile, yatayda çok sayıda alan veya alias kullanılabilir:

- Örneğin aynı kök alan için `friends1, friends2, ..., friends100` şeklinde alias'larla tekrar tekrar istek yapılması,
- Tek HTTP isteği içinde onlarca GraphQL sorgusunun *batch* hâlinde gönderilmesi gibi durumlar, arka uç veri kaynaklarında ciddi yük oluşturabilir.

Bu yüzden çoğu GraphQL sunucusunda:

- Tek dokümandaki *top-level field* sayısı,
- Kullanılabilen *field alias* sayısı,
- Tek batch içinde izin verilen maksimum sorgu (*operation*) sayısı

için sınırlar (*limits*) konfigüre edilebilir.

Rate Limiting

Derinlik ve genişlik sınırları, sorgu şeklini kontrol eder; ancak bazı alanlar diğerlerinden çok daha *pahalı* olabilir. Örneğin, karmaşık rapor üreten bir alan, sıradan bir name alanına göre çok daha fazla CPU ve IO kullanabilir.

Bu nedenle:

- Yalnızca toplam HTTP istek sayısına bakmak yerine,
- İş katmanında, alan bazlı ve kullanıcı bazlı *rate limit* politikaları uygulanması,
- Örneğin bir kullanıcının belirli bir zaman penceresinde kaç adet “pahalı sorgu” çalıştırabileceğinin sınırlandırılması

genellikle daha anlamlıdır.

Query Complexity Analysis

Şema üzerindeki tiplere ve alanlara birer *maliyet* (ağırlık) atanarak, gelen sorgunun tahmini *karmaşıklık puanı* hesaplanabilir:

- Basit scalar alanlar düşük maliyetli,
- Büyük join’ler veya pahalı hesaplamalar yüksek maliyetli kabul edilebilir.

Daha sonra:

- Toplam maliyet belirlenen *maksimum sorgu bütçesini* aşıyorsa, istek tamamen reddedilebilir.
- Veya bu maliyet, rate limiting hesabına dahil edilerek istemcinin belirli bir zaman dilimindeki toplam *query bütçesinden* düşülebilir.

Şema ve Argüman Güvenliği

Argümanların Doğrulanması ve Temizlenmesi

GraphQL tip sistemi, bir argümanın temel tip uyumluluğunu (örneğin `String`, `Int`) doğrular; ancak içeriğin güvenli olduğunu garanti etmez. Örneğin:

- `String` tipindeki bir alan zararlı HTML veya script içerebilir.

Bu nedenle:

- Kullanıcı girişlerinin, resolver içinde veya iş katmanında sanitize edilmesi,

- Gerekirse özel scalar tiplerinin (Email, URL, SafeHTML vb.) tanımlanması

önerilir.

Introspection

Introspection, GraphQL şemasının kendisinin sorgulanmasını sağlayan güçlü bir özelliktir. Ancak yalnızca *first-party* istemcilere hizmet veren API'lerde, üretim ortamında introspection'ı kısmen veya tamamen kapatmak tercih edilebilir.

Yine de:

- Introspection'ı kapatmanın tek başına tam bir güvenlik sağlamadığı,
- Sadece şemanın keşfedilmesini biraz daha zorlaştırdığı,
- Asıl korumanın *trusted documents*, *doğru authorization ve demand control mekanizmaları* ile sağlanması gerektiği

unutulmamalıdır.

Hata Mesajlarının Maskeleyesi

Geliştirme ortamında:

- Yanlış alan adı yazıldığında hangi alanların mevcut olduğunu söyleyen detaylı hata mesajları,
- Validation ve execution hatalarıyla ilgili açıklayıcı çıktılar, geliştirici deneyimini iyileştirir.

Ancak üretim ortamında bu detaylar:

- Şemanın yapısı hakkında gereğinden fazla ipucu verebilir,
- Sunucu veya veri kaynakları hakkında hassas teknik bilgi sızdırabilir.

Bu nedenle:

- Production ortamında hata mesajlarının sadeleştirilmesi,
- Teknik detayların ve iç stack trace'lerin gizlenmesi,
- Hem istek doğrulama hatalarında, hem de field execution sırasında oluşan beklenmedik hatalarda kullanıcıya sınırlı bilgi sunulması

önemlidir.

Kimlik Doğrulama ve Yetkilendirme

GraphQL’de kimlik doğrulama (*authentication*) ve yetkilendirme (*authorization*) konuları, diğer API türleri kadar kritiktir. İstemcinin:

- Kim olduğunun güvenilir bir şekilde belirlenmesi,
- Hangi alanlara ve işlemlere erişmesine izin verildiğinin iş katmanında net kurallarla tanımlanması

geniş kapsamlı bir güvenlik stratejisinin vazgeçilmez parçalarıdır.

Genel Özet

GraphQL API’lerinin güvenliği için öne çıkan noktalar şunlardır:

- Seçilen transport protokolü (HTTP, WebSocket vb.) için genel güvenlik uygulamaları (HTTPS, timeout, cache politikaları) GraphQL için de aynen geçerlidir.
- *Demand control* kapsamında:
 - Trusted documents ile allowlist bazlı güvenilir sorgu listesi oluşturmak,
 - Liste alanlarını sayfalararak tek istekte dönebilecek veri miktarını sınırlandırmak,
 - Sorgu derinliği, genişliği ve batch büyüklüğü için limitler koymak,
 - Gelişmiş senaryolar için rate limiting ve query complexity analysis uygulamaksunucu kaynaklarını korumaya yardımcı olur.
- Şema tasarımı ve iş katmanında argüman doğrulama, sanitize etme ve özel scalar tipleri kullanmak, kullanıcı verisini daha güvenli işlenebilir hale getirir.
- Introspection’ı kapatmak ve hata mesajlarını maskelemek, şemanın keşfedilmesini zorlaştırabilir; ancak tek başına yeterli değildir. Bunlar, trusted documents ve güçlü authorization kurallarıyla birlikte kullanılmalıdır.

GraphQL Federation

Genel Bakış

GraphQL federation, büyük bir GraphQL API'yi tek bir monolit olarak tutmak yerine, birden fazla bağımsız GraphQL servisine (*subgraph*) bölüp bunları bir *gateway* üzerinden birleştiren mimari bir yaklaşımdır.

Klasik microservice yaklaşımına benzer şekilde:

- Her domain için ayrı bir servis ve ayrı bir şema vardır,
- Bu servisler bağımsız geliştirilip deploy edilebilir,
- İstemci tarafında ise hâlâ *tek bir birleşik GraphQL API* görünür.

Federation Nedir?

Federation, dağıtık sistemleri organize etme biçimidir. Temel fikir:

- Her bileşen (servis / ekip) kendi alanından sorumludur,
- Kendi kararlarını verebilir, kendi hızında gelişir,
- Ortak ihtiyaçlar için bunların üzerinde bir koordinasyon katmanı bulunur.

Devlet benzetmesiyle:

- Eyaletler: Kendi iç kuralları olan bağımsız alanlar,
- Federal yapı: Eyaletleri bir arada tutan, dışarıya tek ses olarak çıkan katman.

GraphQL federation, bu fikri GraphQL API'lerine uygular.

Federated GraphQL

Federated bir GraphQL mimarisinde:

- **Subgraph:**
 - Kendi GraphQL şemasına ve resolver'larına sahip bağımsız servis.
 - Örneğin: *Users*, *Orders*, *Products* gibi domain bazlı servisler.
- **Gateway:**
 - İstemci ile subgraph'lar arasındaki tek giriş noktasıdır.

- Tüm subgraph şemalarını birleştirerek tek bir *unified schema* sunar.
- Gelen sorguyu alt parçalara bölerek ilgili servislere yönlendirir ve sonuçları tek bir yanıt hâlinde birleştirir.

Örnek bir unified şema:

```
type User {
  id: ID!
  name: String!
  email: String
  orders: [Order!]!
}

type Order {
  id: ID!
  products: [Product!]!
  total: Float!
}

type Product {
  id: ID!
  title: String!
  price: Float!
  inStock: Boolean!
}
```

Bu şemada User, Order ve Product tipleri tek bir grafik gibi görünse de, gerçekte farklı subgraph'lar tarafından sağlanıyor olabilir.

Sorgu Akışı Örneği

İstemcinin gateway'e gönderdiği örnek sorgu:

```
query {
  user(id: "123") {
    name
    orders {
      id
      products {
        title
        price
      }
    }
  }
}
```

```
}  
}  
}
```

Bu akış şu şekilde çalışır:

- `user(id: "123")` alanı *Users* subgraph tarafından çözülür.
- Kullanıcının sipariş bilgileri için *Orders* subgraph'a gidilir.
- Siparişteki ürünler için *Products* subgraph'a gidilir.
- Gateway, tüm bu ara sonuçları toplayıp tek bir JSON yanıt içinde istemciye geri döner.

İstemci, arkada kaç servis çağrısı yapıldığını bilmek zorunda değildir; sadece tek GraphQL endpoint'i ile konuşur.

Avantajlar

Domain-Driven Development

- Her ekip, kendi domain'inin şemasını ve iş kurallarını yönetir (örneğin *Users*, *Orders*, *Payments*).
- Bağımsız geliştirme ve bağımsız deployment mümkündür.
- Ekipler arasında koordinasyon yükü azalır, hız artar.

Servis Bütünlüğü ve Schema Composition

- Schema composition aşaması, subgraph şemalarını birleştirirken:
 - Tip çakışmalarını,
 - Uyuşmayan alan tanımlarını,
 - Kırık referanslarıyakalayabilir.
- Böylece bir ekibin yaptığı hatalı şema değişikliği, diğer ekiplerin alanlarını bozmadan tespit edilebilir.

Ölçeklenebilirlik ve Performans

- Her subgraph bağımsız ölçeklenebilir:
 - Çok okunan *Product* servisi daha fazla replika ile,
 - Daha az kullanılan *Order* servisi daha az kaynakla çalıştırılabilir.
- Gateway, cache ve diğer optimizasyonlarla genel performansı iyileştirebilir.

Tek Birleşik API

- Tüm dağıtık karmaşıklığa rağmen, istemci tarafında:
 - Tek endpoint (`/graphql` gibi),
 - Tek birleşik şemagörünür.
- Frontend ekipleri, arkadaki microservice topolojisini bilmek zorunda kalmadan çalışabilir.

Ne Zaman Federation Kullanmalı?

GraphQL federation özelliklerle:

- Domain-Driven Design (DDD) prensipleriyle çalışan,
- Birden fazla ekip ve domain'e bölünmüş büyük organizasyonlarda,
- Farklı diller ve teknolojiler kullanan microservice mimarilerinde

doğal bir uyum sağlar.

Ancak:

- Gateway, schema registry ve subgraph entegrasyonları için ciddi altyapı ve operasyon maliyeti gerekir.
- Küçük ekipler veya basit domain'ler için gereksiz derecede karmaşık olabilir.

Bu nedenle sık önerilen yaklaşım:

- Önce **monolitik bir GraphQL API** ile başlamak,
- Ekipler ve domain'ler büyüdükçe, ihtiyaç ortaya çıktığında kademeli olarak **federation** mimarisine geçmektir.

Başlarken: Adım Adım

GraphQL federation'a geçişte tipik adımlar:

1. Servis sınırlarını belirle:

- Hangi alanlar ayrı domain olarak ayrılmalı?
- Örneğin: *Users*, *Catalog*, *Orders*, *Payments*.

2. Şemaları tasarla:

- Her domain için tipler ve ilişkiler tanımlanır.
- Domain sınırları şema üzerinde de net olmalıdır.

3. Subgraph'ları uygula:

- Her domain için ayrı bir GraphQL servisi geliştirilir.

4. Gateway kur:

- Tüm subgraph'ları birleştiren federated gateway devreye alınır.

5. Schema registry kullan:

- Subgraph şemaları kayıt altına alınır,
- Schema composition ve uyumluluk kontrolleri bu katmanda yönetilir.

Mevcut monolitik bir GraphQL API'niz varsa:

- Önce bunu ilk subgraph gibi düşünebilir,
- Sonra yavaş yavaş belirli domain'leri ayrı subgraph'lara bölerek federated yapıya geçebilirsiniz.

Common HTTP Errors and Debugging in GraphQL over HTTP

Genel Bakış

GraphQL'i HTTP üzerinden kullanırken, özellikle geliştirme aşamasında çeşitli HTTP hata kodlarıyla karşılaşmak yaygındır. Bu bölümde:

- Sık görülen HTTP ve GraphQL hata kodlarının ne anlama geldiği,
- Yaygın nedenleri,
- Nasıl debug edilebilecekleri,
- Ve GraphQL yanıtları için önerilen medya tipi (`application/graphql-response+json`)

özetlenmektedir.

400 Bad Request (İstek Hatalı / Parse Hatası)

Anlamı. Sunucu, isteği parse edememiştir. Ya JSON body geçersizdir ya da GraphQL sorgu metni sentaks olarak hatalıdır.

Yaygın Nedenler.

- JSON body sentaks hataları (eksik süslü parantez, tırnak hatası vb.),
- "query" alanını içeren bir JSON yerine düz string göndermek (örneğin sadece "{ hero { name } }" göndermek),
- Uygulamanın desteklemediği bir Content-Type kullanmak (örneğin `text/plain` veya `application/graphql`).

Debug Önerileri.

- JSON body'yi bir formatter veya linter ile doğrula,
- POST isteğiyle birlikte `Content-Type: application/json` gönderildiğinden emin ol,
- GraphQL sorgusunu bir IDE veya linter ile sentaks açısından kontrol et.

405 Method Not Allowed (Yanlış HTTP Metodu)

Anlamı. İstek, bu endpoint için desteklenmeyen bir HTTP methodu ile gönderilmiştir. Çoğu GraphQL sunucusu:

- Mutations için **POST**,
- Queries için **POST** (opsiyonel olarak GET)

bekler.

Yaygın Nedenler.

- PUT veya DELETE ile GraphQL isteği göndermek,
- Mutation'ı GET ile göndermek,
- Sunucu yalnızca POST desteklerken GET kullanmak.

Debug Önerileri.

- HTTP methodunu kontrol et:
 - Mutations için her zaman POST,
 - Queries için sunucunun GET desteği olup olmadığını kontrol et.
- Kullandığın sunucunun veya kütüphanenin dokümantasyonuna bakarak hangi methodlara izin verildiğini doğrula.

500 Internal Server Error (Sunucu Hatası)

Anlamı. İstek sunucuya ulaşmış, ancak backend içinde beklenmeyen bir hata oluşmuştur.

Yaygın Nedenler.

- Resolver içinde yakalanmamış exception,
- Sunucu açılışında şema veya konfigürasyon hataları,
- Eksik veya yanlış yapılandırılmış middleware (auth, body parser vb.).

Debug Önerileri.

- Sunucu loglarını ve stack trace'leri incele,
- Resolver fonksiyonlarında hata yakalama ve logging stratejileri uygula,
- Uygulamanın başlatma aşamasında şema, middleware zinciri ve diğer konfigürasyonları doğrula.

200 OK + **errors** (HTTP Başarılı, GraphQL Hatalı)

Anlamı. HTTP isteği başarılıdır (200), ancak GraphQL operasyonu sırasında hata oluşmuştur. Yanıt tipik olarak şu şekildedir:

```
{
  "data": { ... } veya null,
  "errors": [ ... ]
}
```

İki Temel Durum.

- **data var, errors var:**
 - Sorgu şemaya göre geçerlidir,
 - Ancak çalışma sırasında hata oluşmuştur (örneğin hatalı input, yetki reddi, resolver bug'ı).
- **Sadece errors var, data yok:**
 - Sorgu validation aşamasında başarısız olmuştur (örneğin var olmayan alanı sorgulama gibi).

Yaygın Validation Hataları.

- Şemada tanımlı olmayan bir alanı sorgulamak:

```
"Cannot query field \"foo\" on type \"Query\"."
```
- Yanlış argüman tipi veya eksik zorunlu argüman,
- Non-leaf tip için selection set eksik/yanlış,
- Aynı dokümanda birden fazla operasyon varken `operationName` belirtilmemesi.

Debug Önerileri.

- Yanıttaki `errors` dizisini dikkatle incele,
- Eğer hata, field veya tip isimleriyle ilgiliyse:
 - Şemayı introspection veya IDE üzerinden gözden geçir,
 - Field, argüman ve tip isimlerini şemaya uygun hâle getir.

Diğer HTTP Kodları (Implementation-Specific)

Bazı GraphQL sunucu implementasyonları, spesifikasyonun açıkça önermediği ek HTTP kodları kullanabilir:

- **415 Unsupported Media Type:**

- İstek, sunucunun anlamadığı bir `Content-Type` ile gönderilmiştir (örneğin `text/plain`).

- **422 Unprocessable Entity:**

- Bazı implementasyonlar, GraphQL validation hataları için `200 + errors` yerine 422 dönebilir.

Bu davranışlar, GraphQL over HTTP taslak spesifikasyonunda standart olarak önerilmez; kullanılan kütüphaneye veya sunucuya göre değişebilir.

GraphQL Yanıt Formatı ve Medya Tipi

Geleneksel olarak GraphQL sunucuları yanıtları:

- `Content-Type: application/json`

ile döndürmüştür. GraphQL over HTTP taslak spesifikasyonu ise, daha spesifik bir medya tipini önermektedir:

- `application/graphql-response+json`

Neden Önemli?

- Bu medya tipi, payload'ın GraphQL response formatına uygun olduğunu açıkça belirtir,
- 2xx olmayan durum kodlarında bile, eğer `application/graphql-response+json` ise, iç yapısının `data/errors/extensions` şeklinde GraphQL standardına uyduğu varsayılabilir,
- Başka proxy veya gateway'ler JSON dönebilir, ancak `application/graphql-response+json` döndürmezler; bu da yanıtın gerçekten GraphQL katmanından geldiğini ayırt etmeyi kolaylaştırır.

İstemci Tarafında Önerilen Kullanım.

- Accept header'ında hem yeni hem eski tipi talep edebilirsiniz:

```
Accept: application/graphql-response+json, application/json;q=0.9
```

- Böylece:
 - Öncelikle `application/graphql-response+json` tercih edilir,
 - Sunucu bunu desteklemiyorsa `application/json` ile devam edilir.

Kod Örneği

0. Önerilen klasör yapısı

text

```
graphql-sql-example/  
├─ schema.sql  
├─ package.json  
└─ src/  
    ├─ db.ts  
    ├─ schema.ts  
    ├─ resolvers.ts  
    └─ server.ts
```

Aşağıdakileri **ayrı ayrı dosyalara** koyduğunu düşün.

schema.sql – DB tarafı

sql

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(200) NOT NULL UNIQUE  
);
```

Şekil 1: GraphQL dokümanı için örnek ekran görüntüsü

2 src/db.ts – PostgreSQL bağlantısı

```
ts

// src/db.ts
import { Pool } from 'pg';

export const pool = new Pool({
  host: 'localhost',
  port: 5432,
  user: 'myuser',
  password: 'mypassword',
  database: 'mydb',
});
```

Şekil 2: GraphQL dokümanı için örnek ekran görüntüsü

3 src/schema.ts – GraphQL şeması (typeDefs)

```
ts

// src/schema.ts
import { gql } from 'apollo-server';

export const typeDefs = gql`
  type User {
    id: ID!
    name: String!
    email: String!
  }

  type Query {
    users: [User!]!
    user(id: ID!): User
  }

  type Mutation {
    createUser(name: String!, email: String!): User!
  }
`;
```

Şekil 3: GraphQL dokümanı için örnek ekran görüntüsü

src/resolvers.ts – Resolver’lar (içinde SQL kullanan katman)

```
ts Copy code

// src/resolvers.ts
import { pool } from './db';

interface CreateUserArgs {
  name: string;
  email: string;
}

interface UserArgs {
  id: string;
}

export const resolvers = {
  Query: {
    users: async () => {
      const result = await pool.query(
        'SELECT id, name, email FROM users ORDER BY id DESC'
      );
      return result.rows;
    },

    user: async (_parent: unknown, args: UserArgs) => {
      const result = await pool.query(
        'SELECT id, name, email FROM users WHERE id = $1',
        [args.id]
      );
      return result.rows[0] ?? null;
    },
  },

  Mutation: {
    createUser: async (_parent: unknown, args: CreateUserArgs) => {
      const result = await pool.query(
        `
        INSERT INTO users (name, email)
        VALUES ($1, $2)
        RETURNING id, name, email
        `,
        [args.name, args.email]
      );
      return result.rows[0];
    },
  },
};
```

Şekil 4: GraphQL dokümanı için örnek ekran görüntüsü

5 src/server.ts – Apollo Server’i başlatan entry point

```
ts

// src/server.ts
import { ApolloServer } from 'apollo-server';
import { typeDefs } from './schema';
import { resolvers } from './resolvers';

const server = new ApolloServer({
  typeDefs,
  resolvers,
});

async function start() {
  const { url } = await server.listen({ port: 4000 });
  console.log(`🚀 GraphQL server ready at ${url}`);
}

start().catch((err) => {
  console.error('Failed to start server:', err);
});
```

Şekil 5: GraphQL dokümanı için örnek ekran görüntüsü

1. Kötü (ve çok yaygın) REST API tasarımı

Diyelim ki mobil ana ekranda şunlara ihtiyacın var:

- Kullanıcı bilgisi → `name`, `avatar`
- Premium durumu → `isPremium`
- Önerilen sorular / date fikirleri → `recommendations`
- Favoriler → `favorites`
- Bildirim özeti → `unreadCount`

"Dağınık" REST API tasarımı şöyle olsun:

- `GET /me`
- `GET /me/premium`
- `GET /me/recommendations`
- `GET /me/favorites`
- `GET /notifications/summary`

Mobil taraf (React Native pseudo-code):

```
ts
Copy code

async function loadHomeScreen() {
  try {
    const [meRes, premiumRes, recRes, favRes, notifRes] = await Promise.all([
      fetch(`${API_URL}/me`),
      fetch(`${API_URL}/me/premium`),
      fetch(`${API_URL}/me/recommendations`),
      fetch(`${API_URL}/me/favorites`),
      fetch(`${API_URL}/notifications/summary`),
    ]);

    const me = await meRes.json();
    const premium = await premiumRes.json();
    const recommendations = await recRes.json();
    const favorites = await favRes.json();
    const notificationsSummary = await notifRes.json();

    setState({
      me,
      isPremium: premium.isPremium,
      recommendations,
      favorites,
      unreadCount: notificationsSummary.unreadCount,
    });
  } catch (e) {
    console.error("Home load error", e);
    // Her API için ayrı error handling vs.
  }
}
```

Sorunlar:

- 5 ayrı network isteği
→ Yavaş / instable mobil ağlarda ciddi gecikme.
- Her endpoint kendi response shape'ine sahip
→ client tarafı mapping ve error handling karmaşık.
- Bir şey daha eklemek istesen (mesela `streak` bilgisi)
→ muhtemelen 6. endpoint veya mevcut endpoint'lere yeni alanlar, versionlama derdi...

Şekil 6: GraphQL dokümanı için örnek ekran görüntüsü

2. Aynı iş GraphQL ile: tek query, tek request

2.1. GraphQL şeması (basitleştirilmiş)

```
graphql

type User {
  id: ID!
  name: String!
  avatarUrl: String
  isPremium: Boolean!
}

type Recommendation {
  id: ID!
  title: String!
  imageUrl: String
}

type FavoriteItem {
  id: ID!
  title: String!
}

type NotificationsSummary {
  unreadCount: Int!
}

type Query {
  me: User!
  homeRecommendations: [Recommendation!]!
  favorites: [FavoriteItem!]!
  notificationsSummary: NotificationsSummary!
}
```

Arka tarafta resolver'lar hâlâ aynı REST/DB çağrılarını yapabilir ama client bunu görmez.

2.2. Mobil home screen GraphQL query'si

```
graphql

query HomeScreenData {
  me {
    id
    name
    avatarUrl
    isPremium
  }
  homeRecommendations {
    id
    title
    imageUrl
  }
  favorites {
    id
    title
  }
  notificationsSummary {
    unreadCount
  }
}
```

Tek endpoint: `POST /graphql`

Tek body: yukarıdaki query.

Şekil 7: GraphQL dokümanı için örnek ekran görüntüsü

2.3. Mobil tarafta (Apollo Client örneđi)

```
import { gql, useQuery } from '@apollo/client';

const HOME_SCREEN_QUERY = gql`
  query HomeScreenData {
    me {
      id
      name
      avatarUrl
      isPremium
    }
    homeRecommendations {
      id
      title
      imageUrl
    }
    favorites {
      id
      title
    }
    notificationsSummary {
      unreadCount
    }
  }
`;

export function useHomeScreenData() {
  const { data, loading, error } = useQuery(HOME_SCREEN_QUERY);

  return {
    loading,
    error,
    me: data?.me,
    recommendations: data?.homeRecommendations ?? [],
    favorites: data?.favorites ?? [],
    unreadCount: data?.notificationsSummary?.unreadCount ?? 0,
  };
}
```

Fark:

- Mobil taraf **tek hook**, **tek ađ isteđi** ile bütün datayı alıyor.
- "Bu ekranda favorilere gerek yok" dersen → query'den **favorites** kısmını silmen yeterli; backend değişmeden request hafifler.
- "Streak" eklemek istedin → **user** tipine **streak** alanını ekleyip sadece isteyen ekranın query'sine koyman yeterli.

Özetle:

- **Kötü REST tasarımı** → 4-5 fetch, bir sürü mapping & error handling.
- Aynı senaryo GraphQL ile → tek **HomeScreenData** query'si, tek request, daha sade client kodu.

Şekil 8: GraphQL dokümanı için örnek ekran görüntüsü